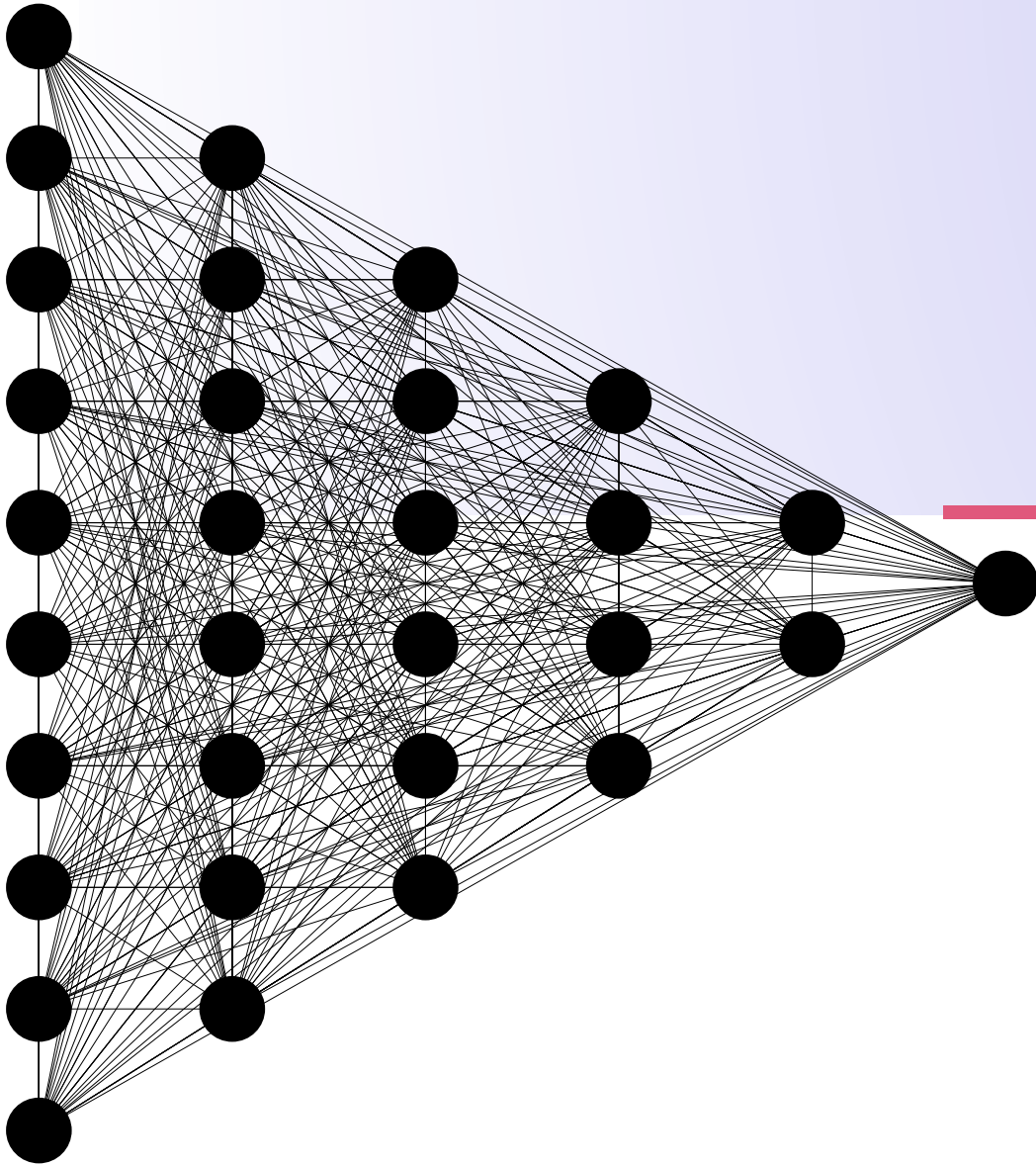


Understanding Neural Networks



Dr. Benoit Mialet

benoit.mialet@hotmail.fr

benoit.mialet@edu.dsti.institute



School of Engineering

From Perceptron to Deep Learning

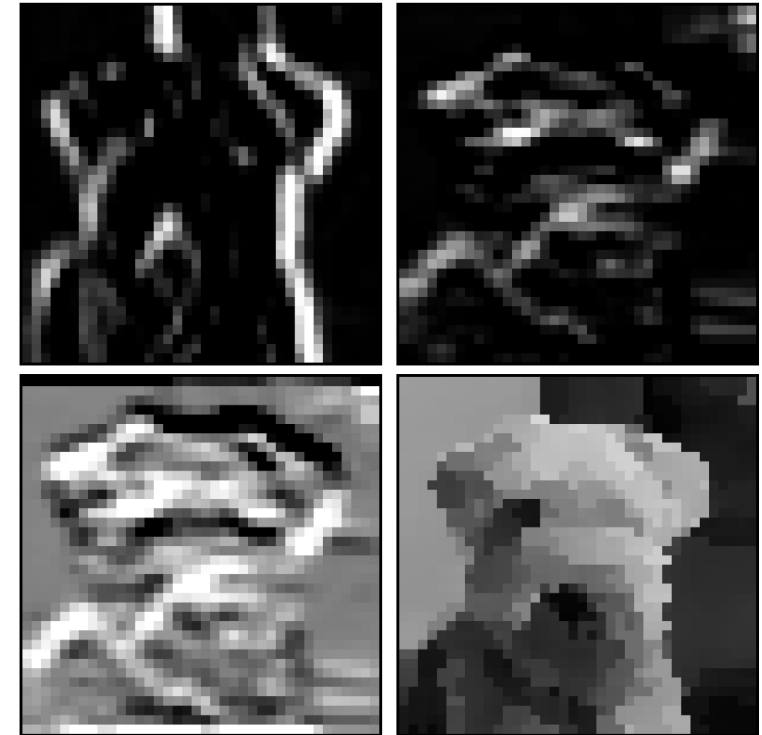
What are neural networks?

- ❑ Artificial Neural Networks (ANNs) or Neural Networks, are...
 - A subset of Machine learning
 - Core element of Deep Learning
 - Used to make all kind of predictions

What are neural networks?

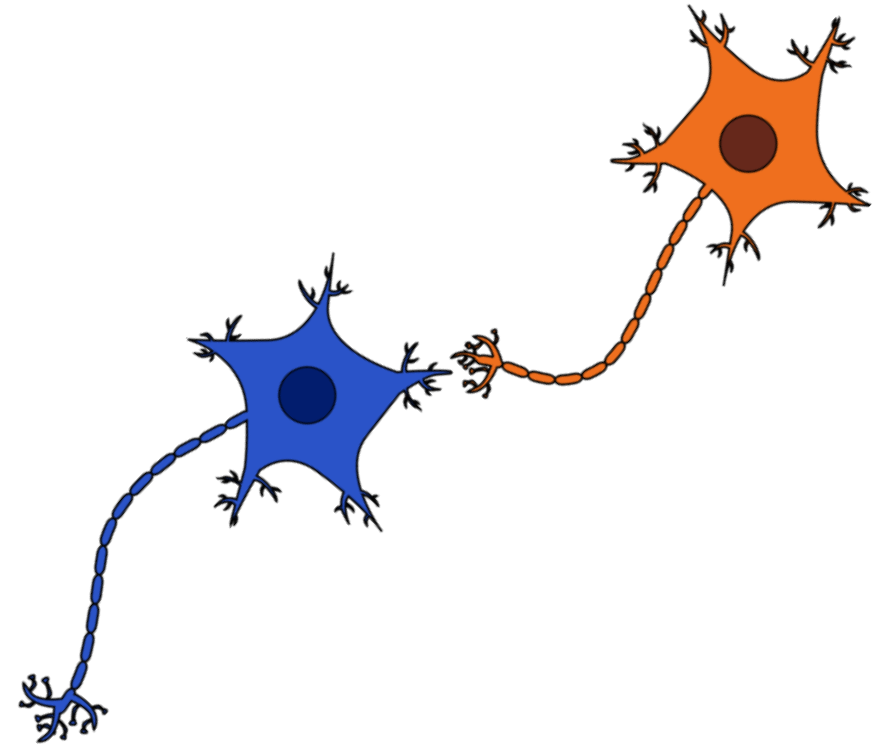
- ❑ Artificial Neural Networks (ANNs) or Neural Networks, are...
 - A subset of Machine learning
 - Core element of Deep Learning
 - Used to make all kind of predictions
 - A **universal approximation algorithm**
 - Able to extract underlying relationships (“**patterns**” or “**features**”) in data

Feature Maps

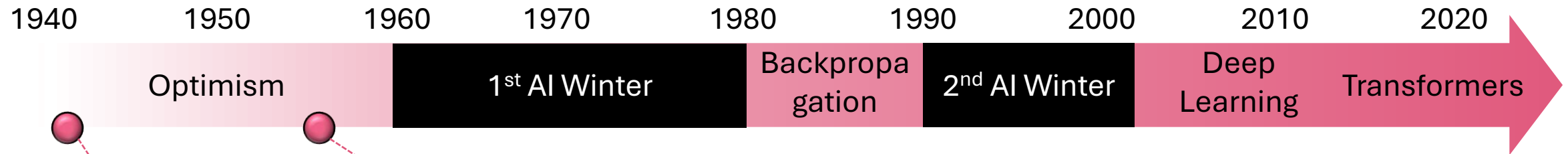


What are neural networks?

- ❑ Artificial Neural Networks (ANNs) or Neural Networks, are...
 - A subset of Machine learning
 - Core element of Deep Learning
 - Used to make all kind of predictions
 - A **universal approximation algorithm**
 - Able to extract underlying relationships (“**patterns**” or “**features**”) in data
 - Based on units called neurons
 - Inspired from human brain functioning
 - Scalable



History of Neural Networks



1943

McCulloch & Pitts

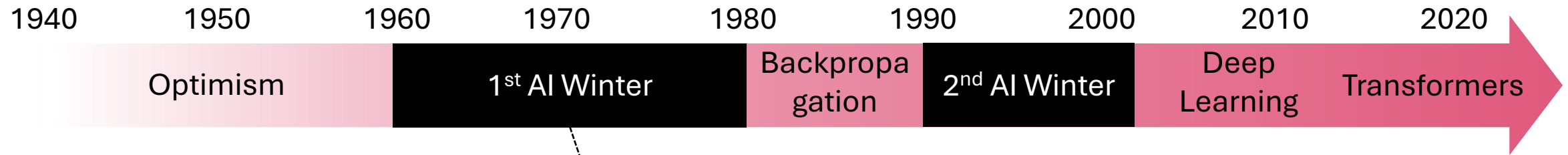
First mathematical
model of an **artificial
neuron**

1957

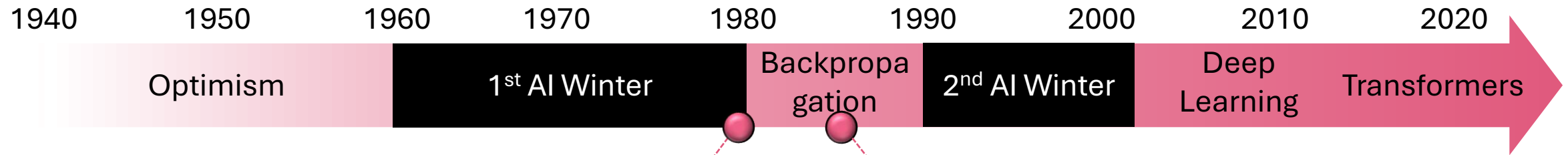
[Rosenblatt](#)

First learning algorithm
for a simple neural
network called
Perceptron

History of Neural Networks



History of Neural Networks



1979

Fukushima

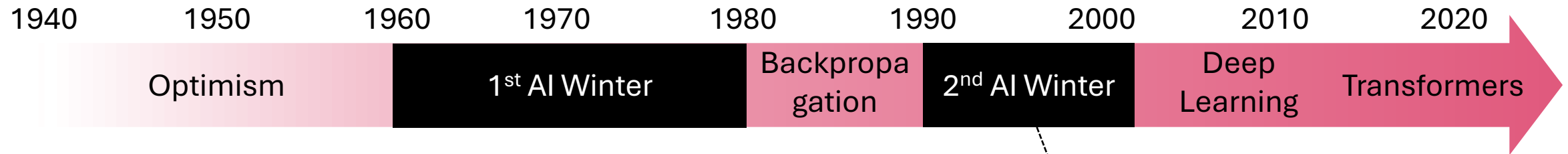
First **Convolutional**
Neural Network

1986

*Rumelhart,
Hinton & Williams*

Back Propagation
improves Neural
Networks

History of Neural Networks

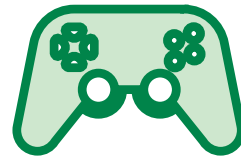
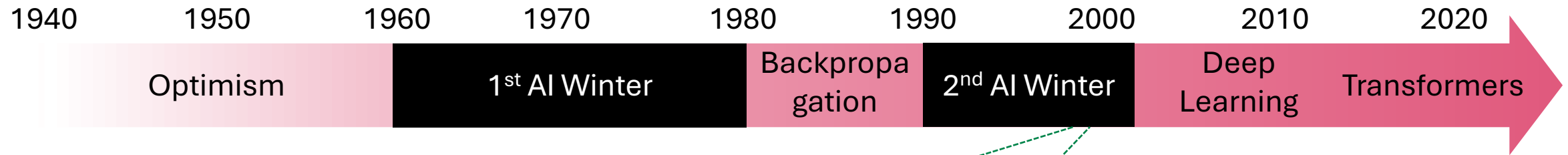


AI Winter

Neural Networks
fall out of favor

Few Data

History of Neural Networks

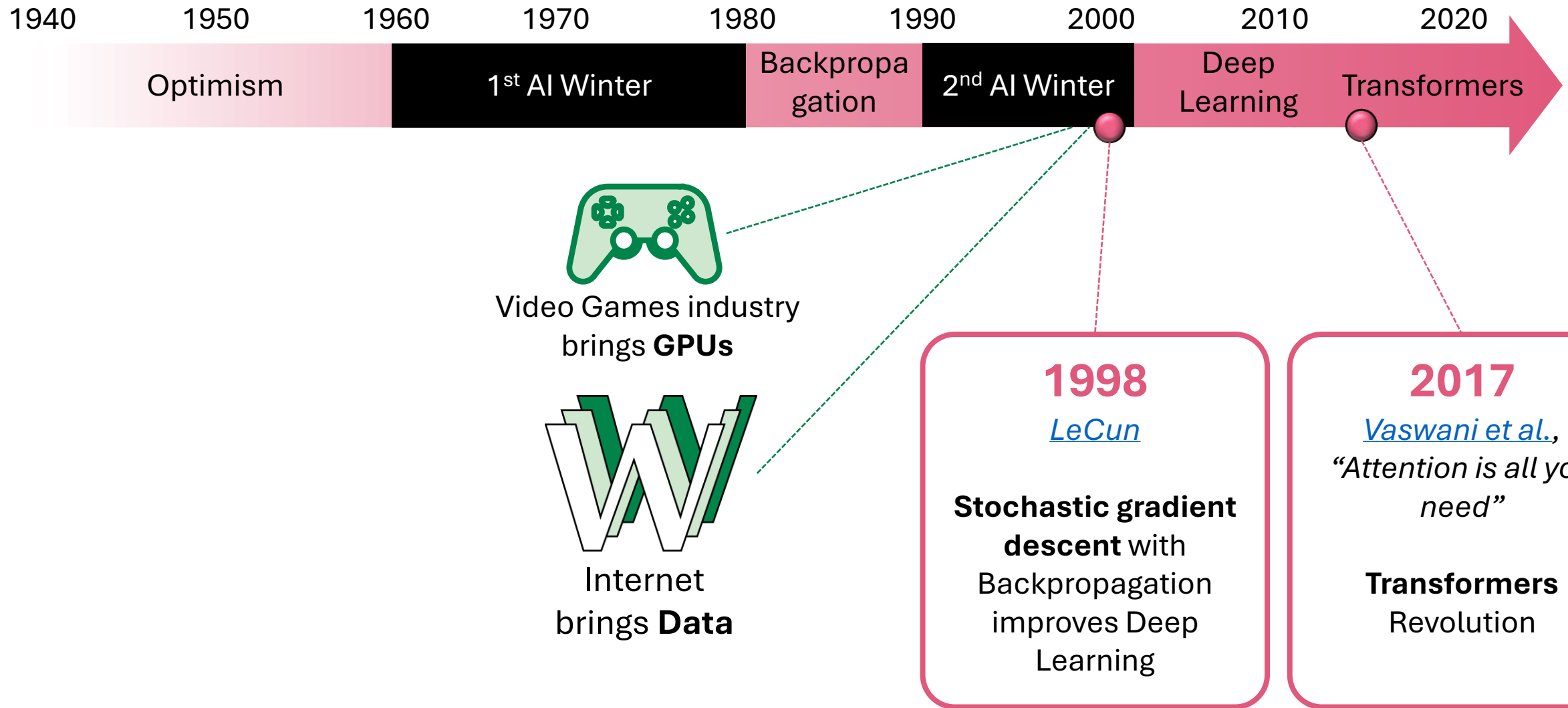


Video Games industry
brings **GPUs**



Internet
brings **Data**

History of Neural Networks



2018 Turing award winners: “Fathers of Deep Learning”



Yann LeCun



Geoffrey Hinton



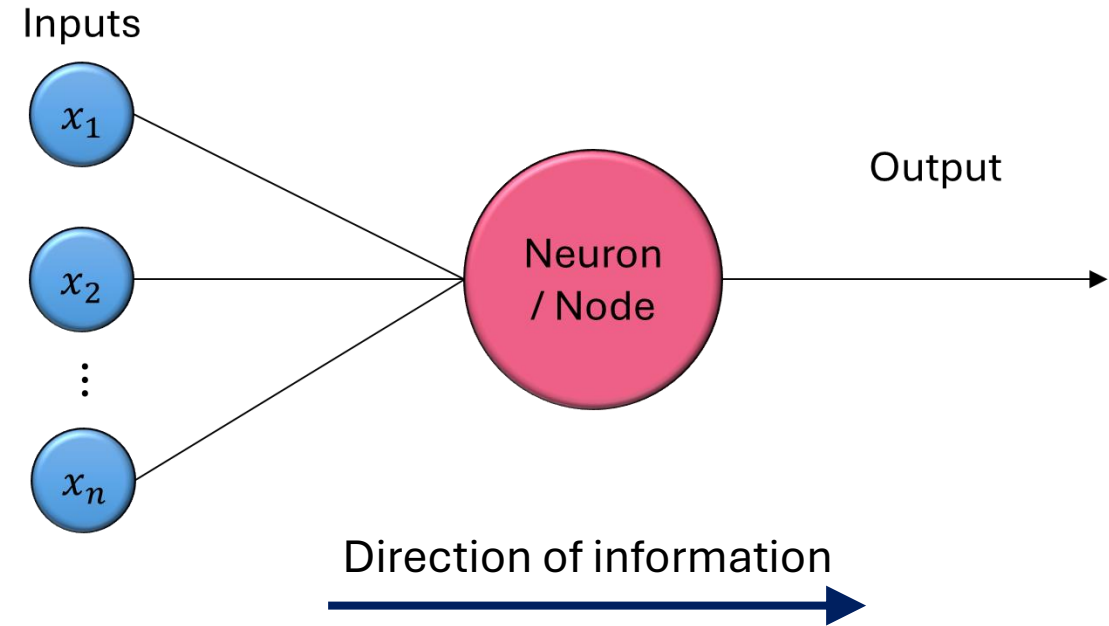
Yoshua Bengio



[“Deep Learning” \(2015\) Nature](#)

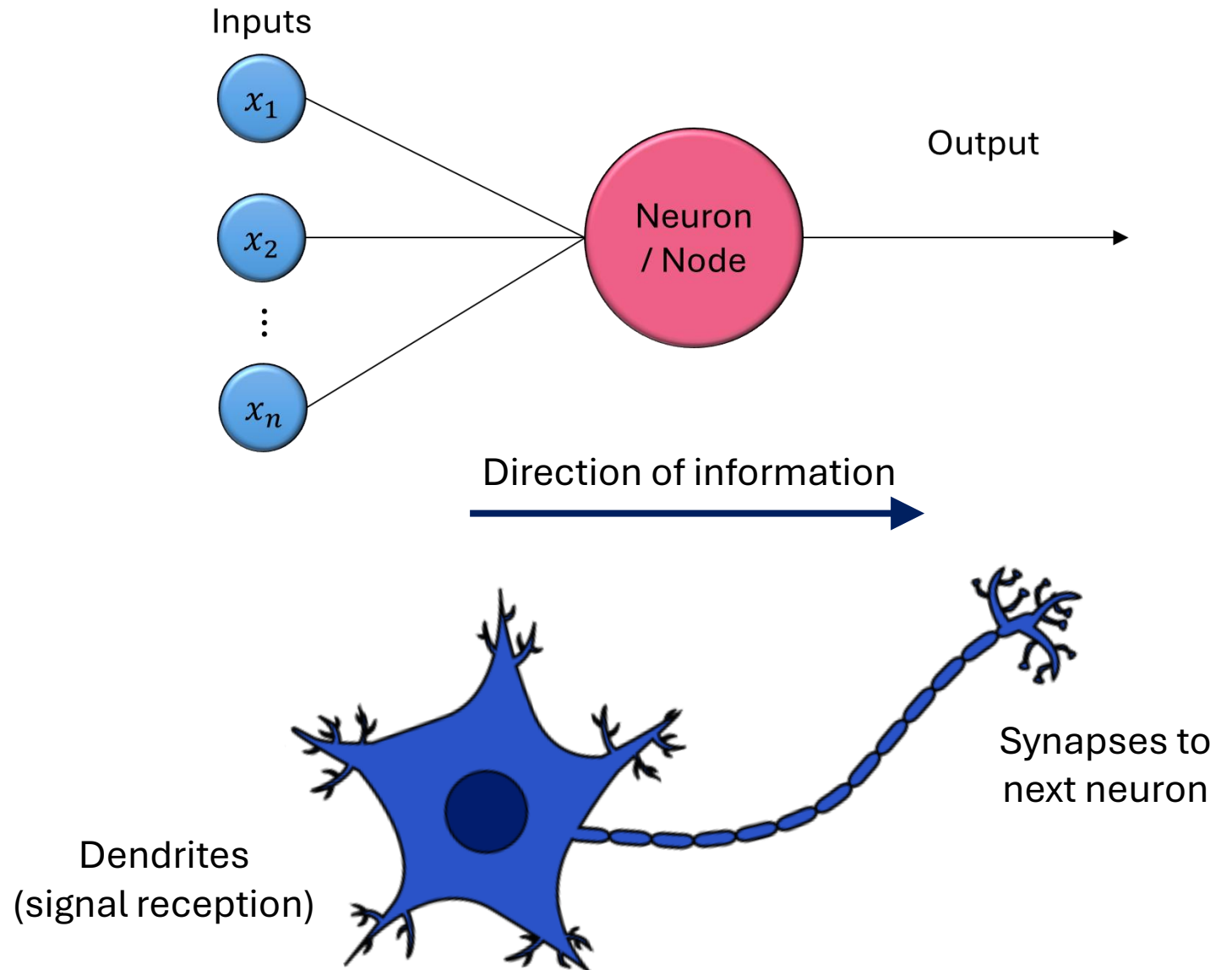
The Artificial Neuron

- ❑ Base unit of a neural network
- ❑ Computational unit that takes inputs and yields outputs
- ❑ Also called “**node**”



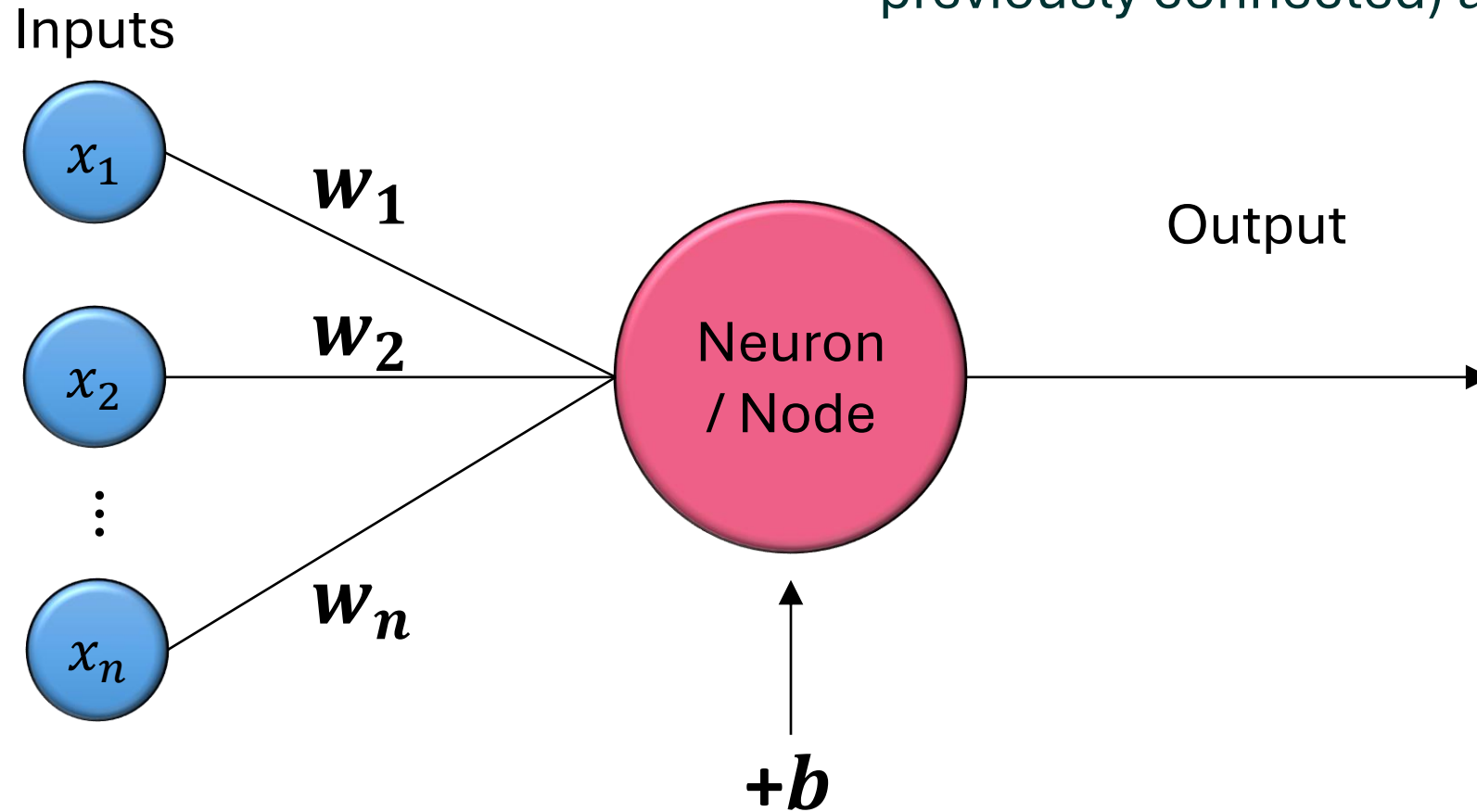
The Artificial Neuron

- ❑ Base unit of a neural network
- ❑ Computational unit that takes inputs and yields outputs
- ❑ Inspired from the base unit of our brain: the neuron

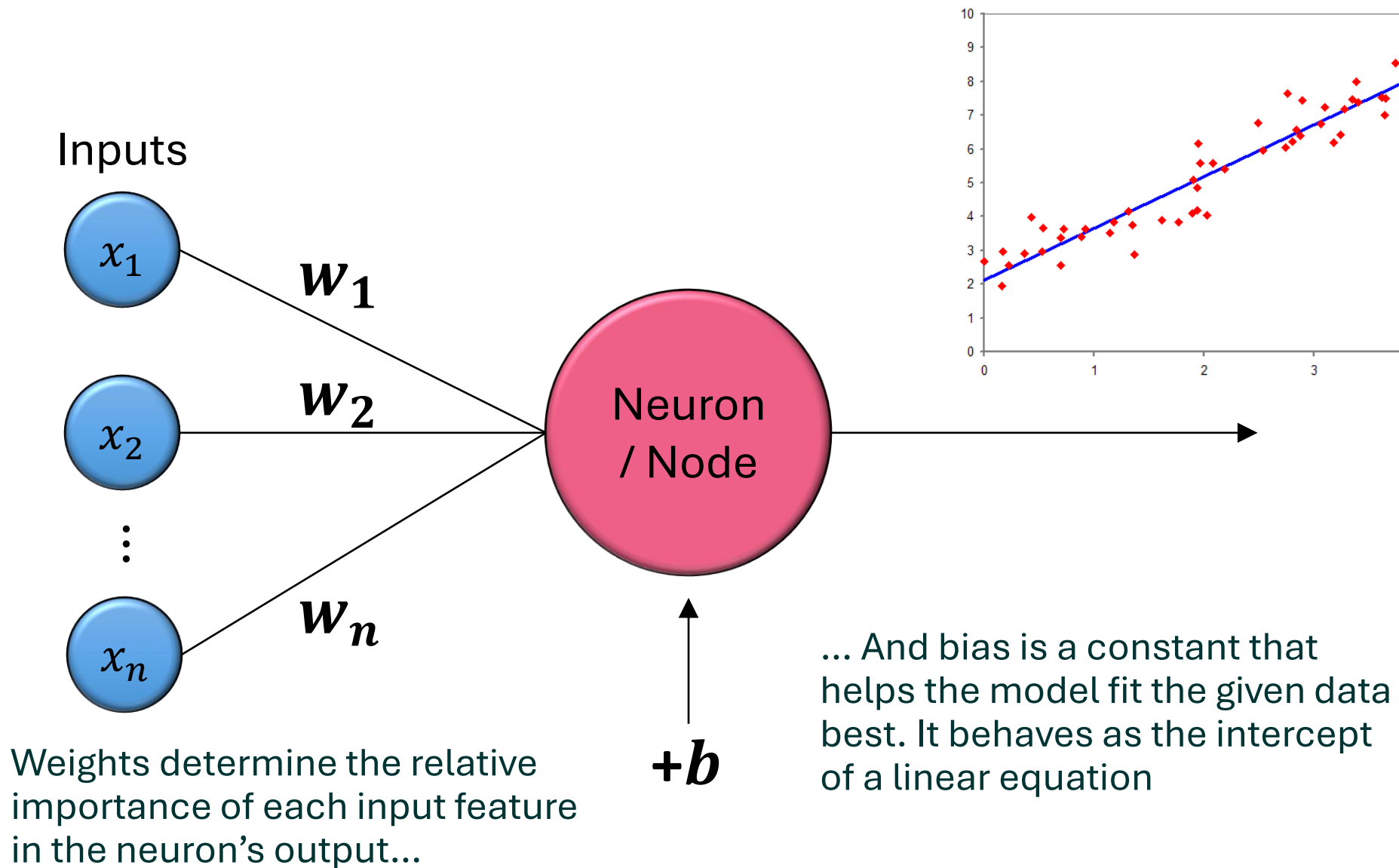


The Artificial Neuron

- An artificial neuron's is composed by a collection of weights (1 per neuron previously connected) and 1 bias



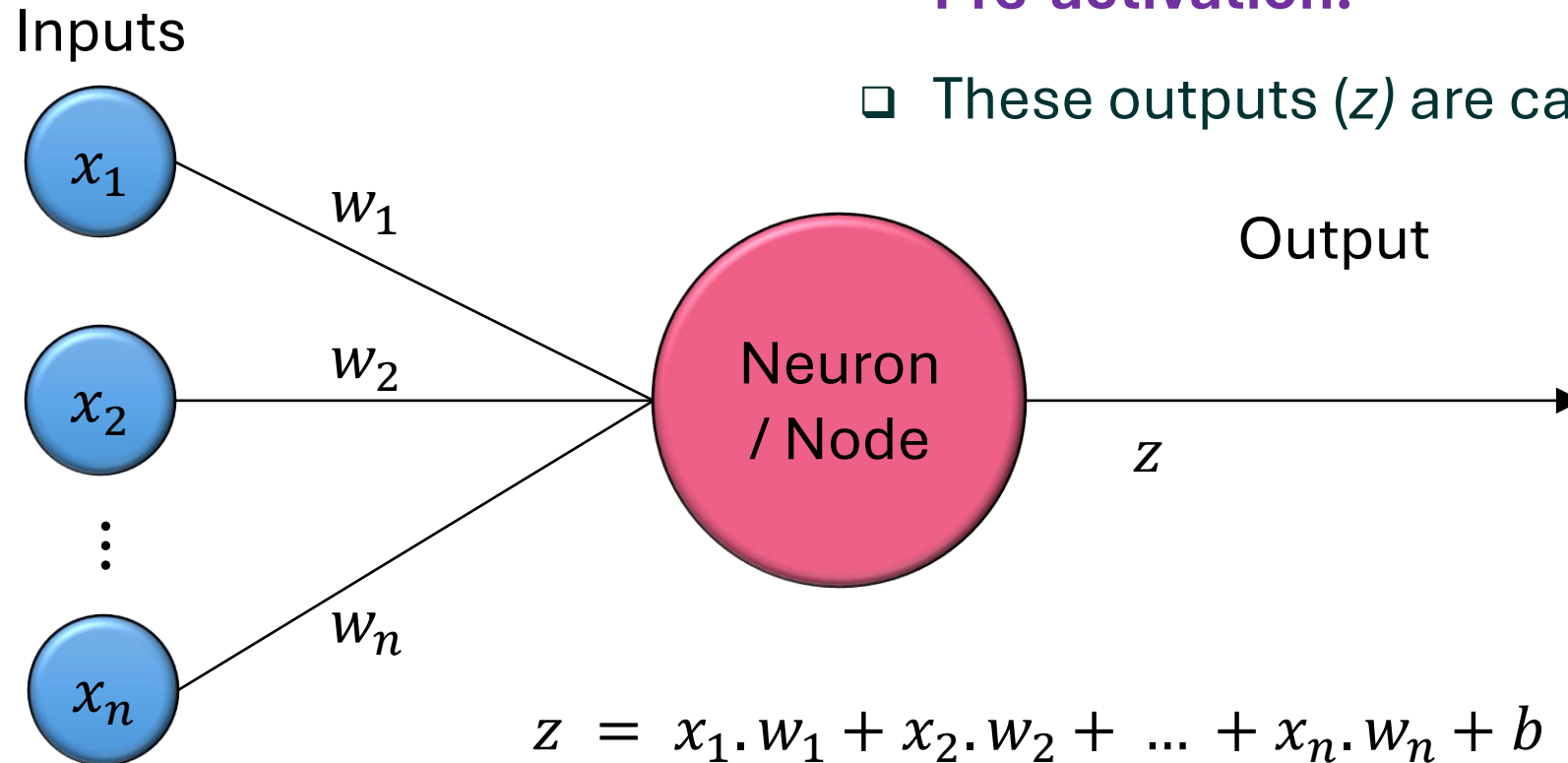
The Artificial Neuron



The Artificial Neuron

- A neuron's output is a **linear combination** of its inputs and weights & bias. This is **Pre-activation**.

- These outputs (z) are called “**Logits**”

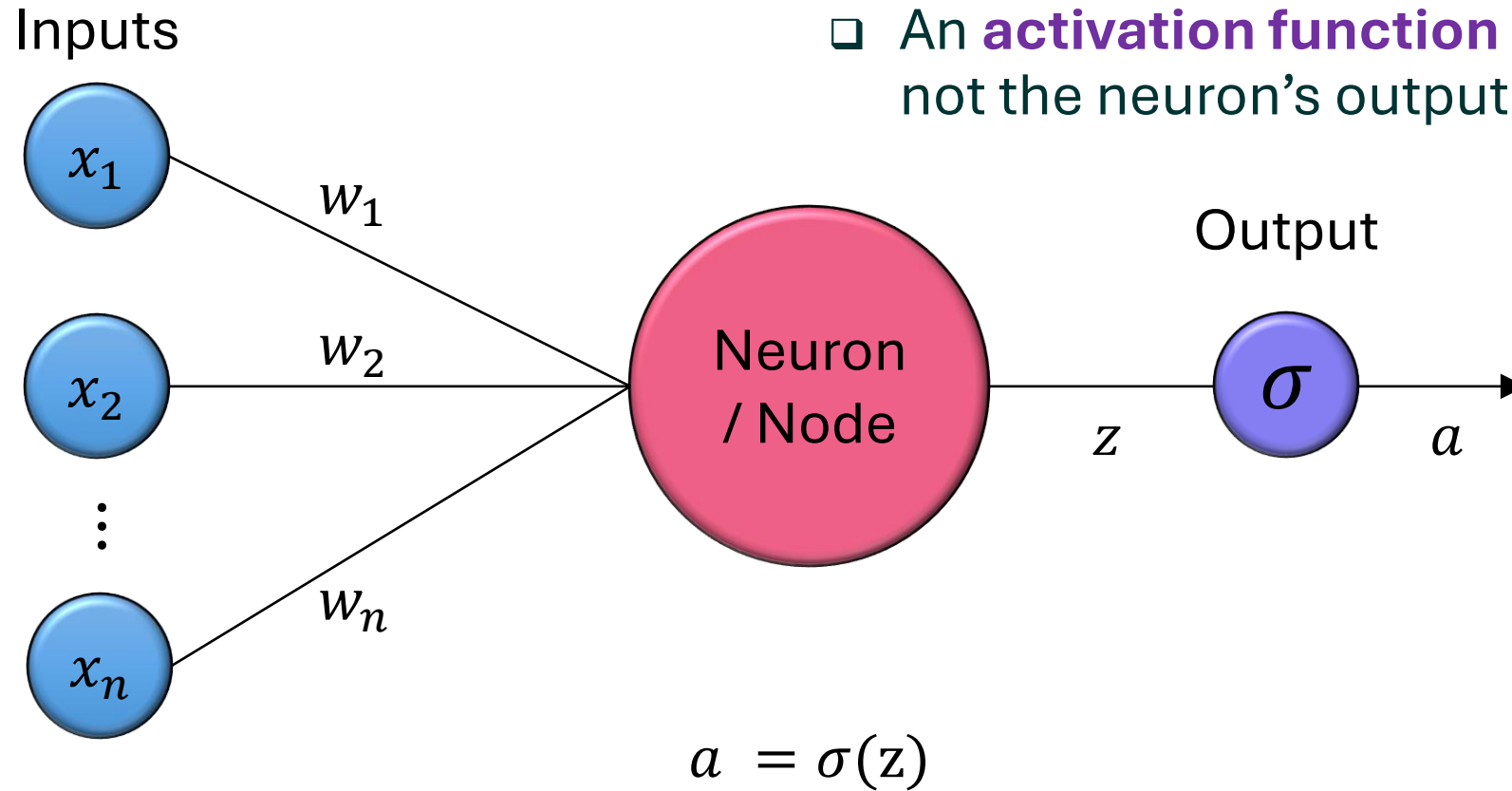


$$z = x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n + b$$

$$z = \sum x_i \cdot w_i + b$$

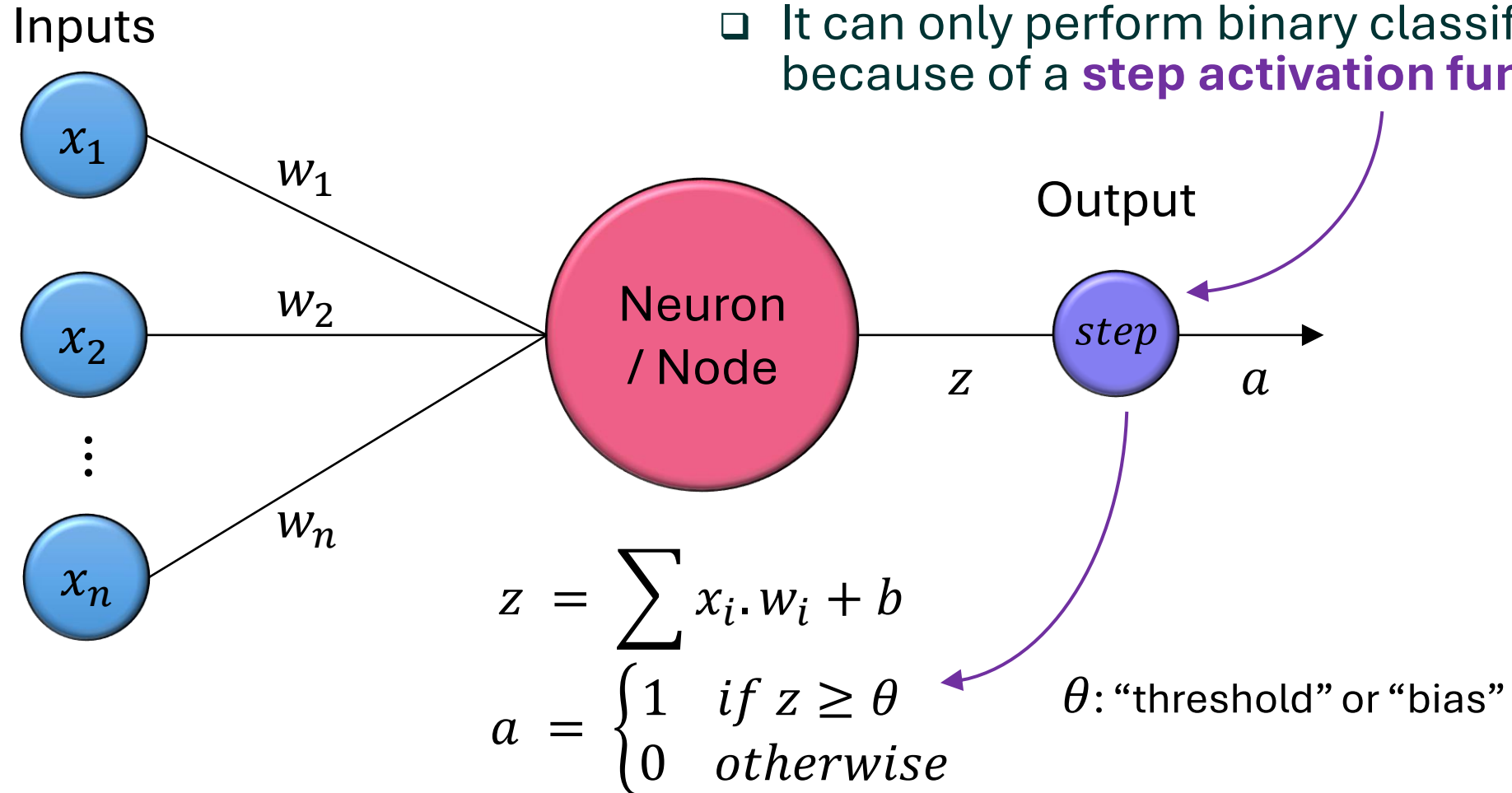
The Artificial Neuron

- ❑ Pre-activation is actually followed by an **activation** step
- ❑ An **activation function** (σ) “activates” or not the neuron’s output



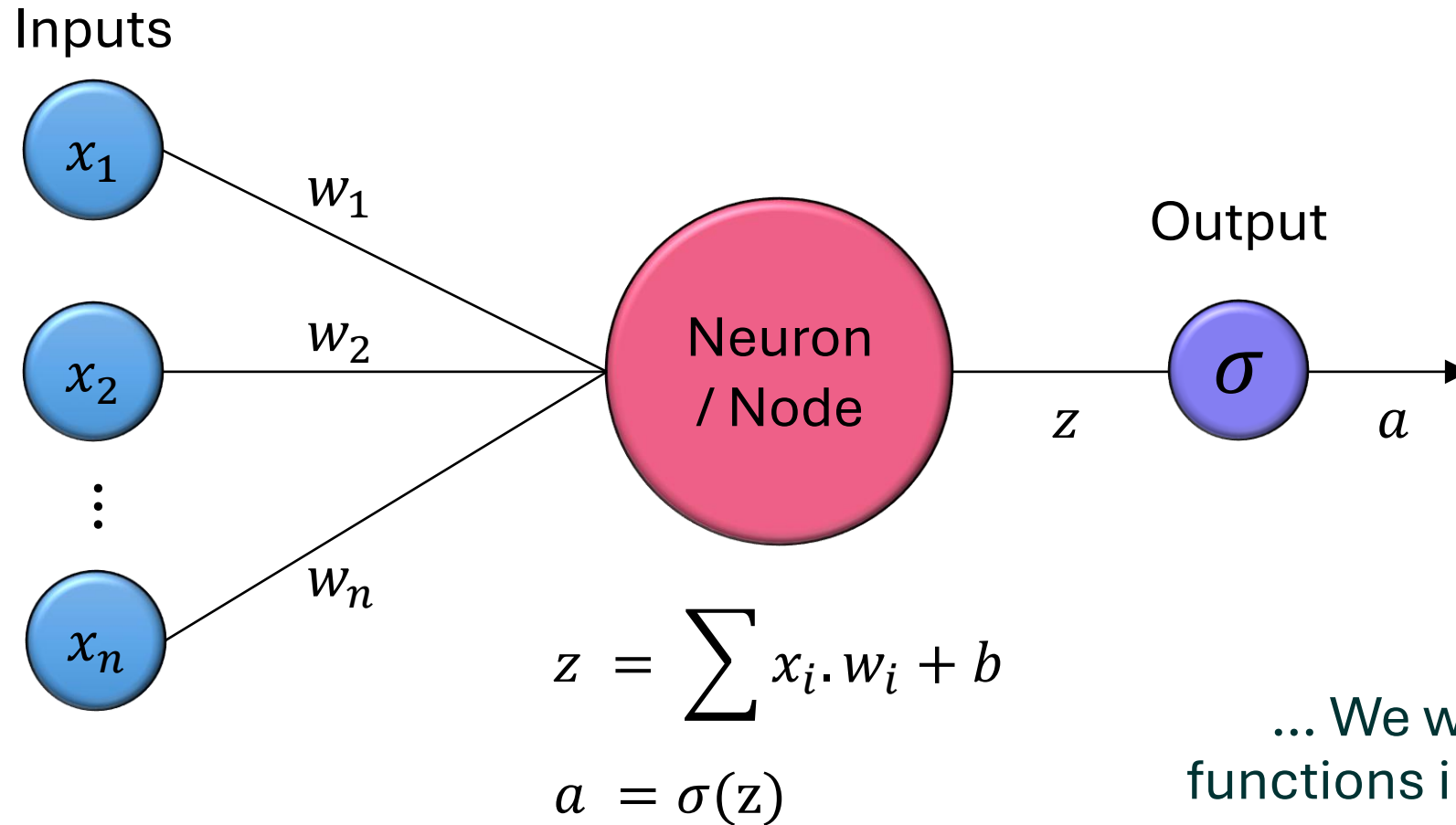
The Perceptron, first artificial neuron

- ❑ The original “**Perceptron**” was the first artificial neuron designed in 1957
- ❑ It can only perform binary classification because of a **step activation function**



The Perceptron, first artificial neuron

- Nowadays, an artificial neuron can use any type of activation function

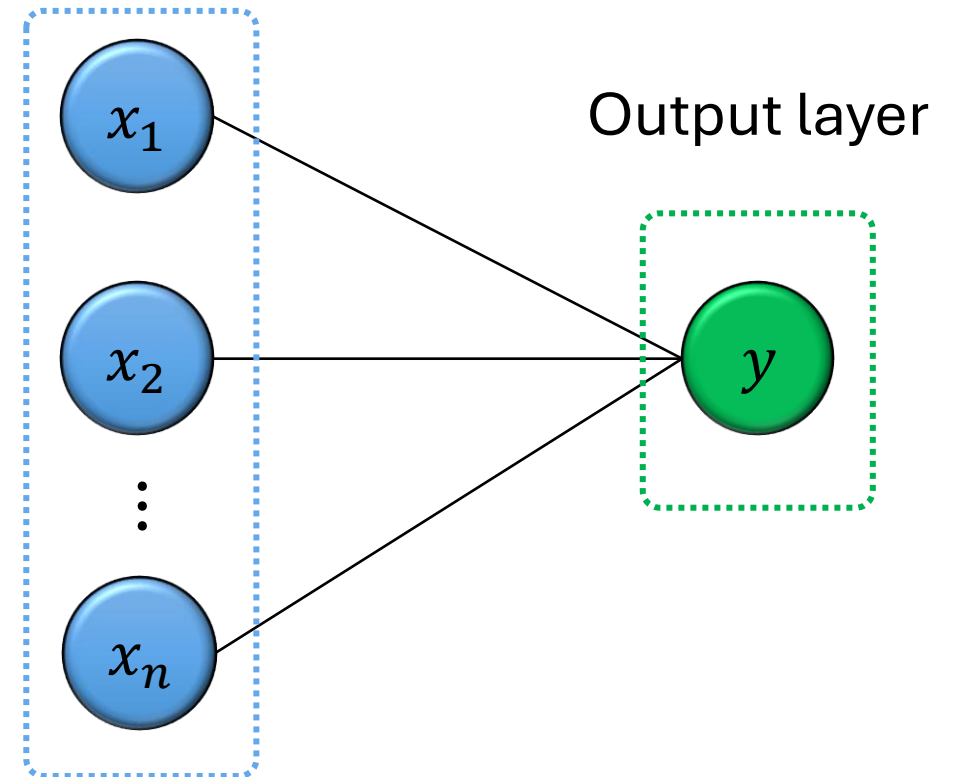


... We will see activation functions in detail later on

The Layer

- ❑ A layer is a group of neurons (**1 to n**) at the same level of a neural network
- ❑ The input layer holds the input values, while the other layers process and transform information
- ❑ FYI, output of a neural network is commonly written “y”

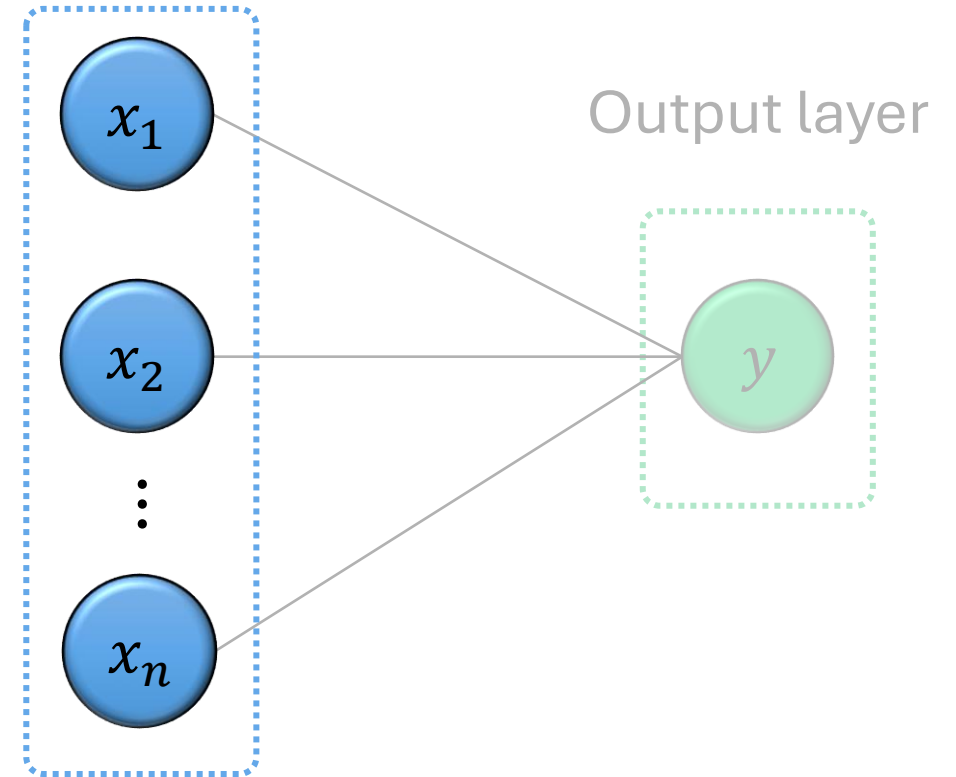
Input layer



The Layer

- ❑ Input layer (or “input features”), correspond to values that a model will directly process
- ❑ They **have to be numerical** (integers, floats)
 - E.g. : area, intensity, speed, counts, etc.

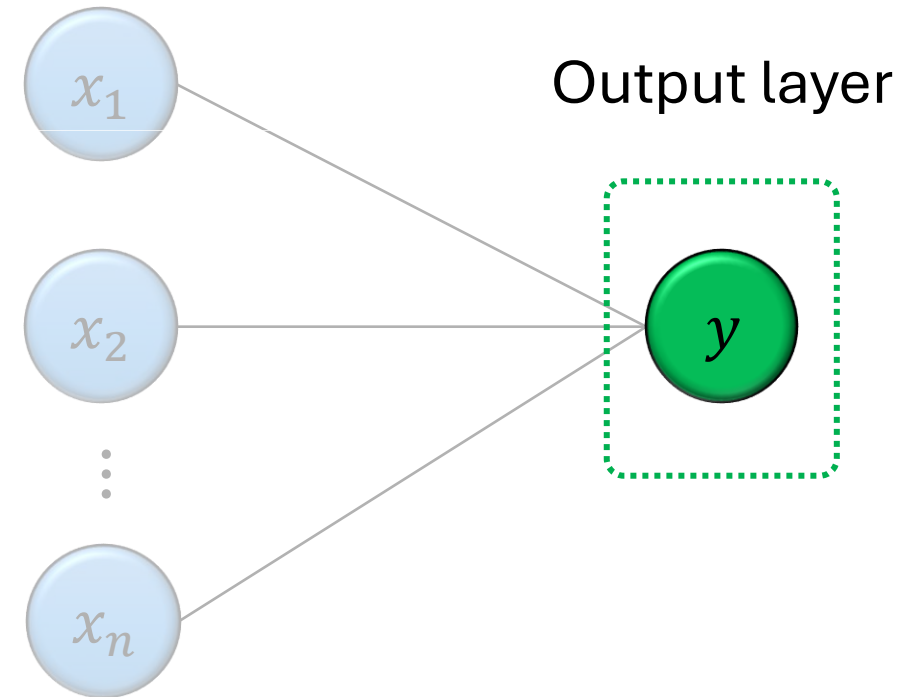
Input layer



The Layer

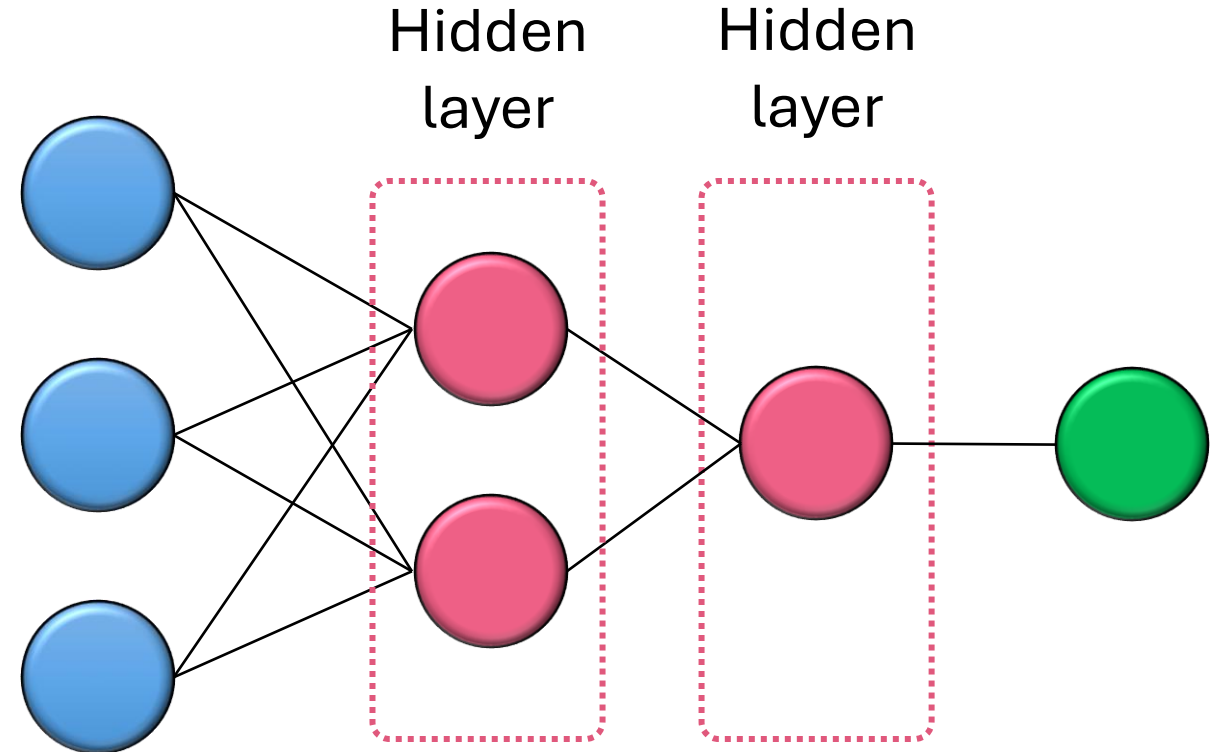
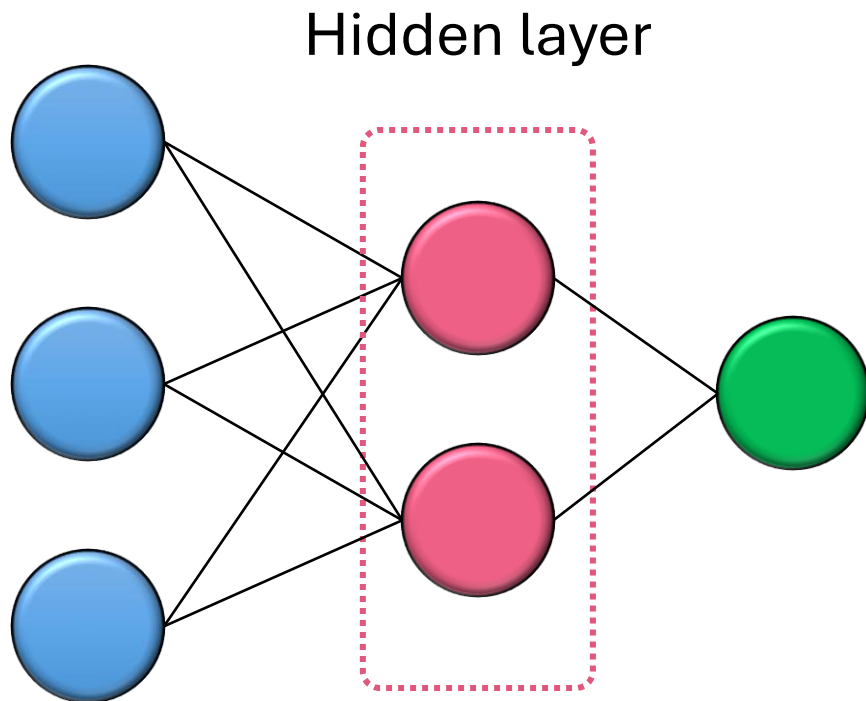
- ❑ The output layer of a neural network produces the final predictions of the node
- ❑ The type of these values depends entirely on the learning task. These values represent either:
 - **continuous predictions** (for regression). Example: -1537.89
 - **or probabilities of classes** (for classification). Examples: 0.157, 0.998, etc.

Input layer



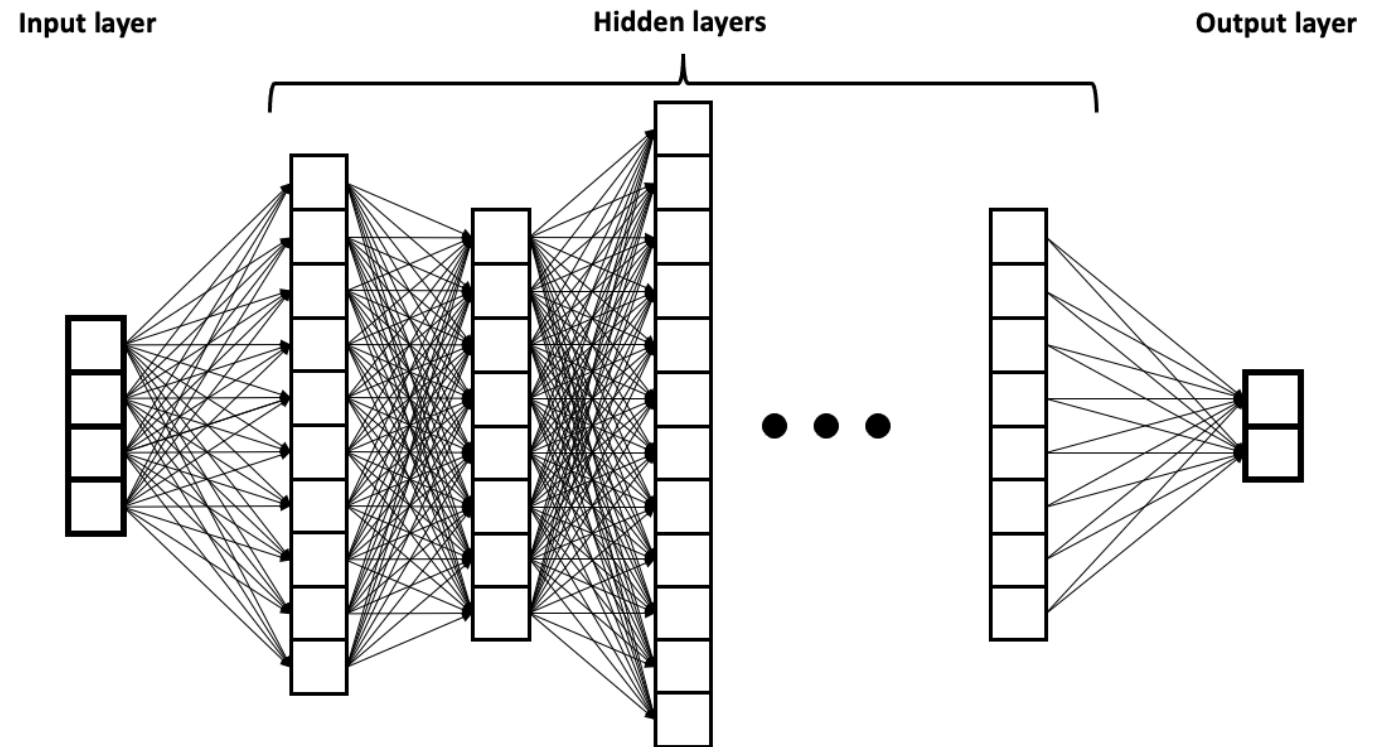
The Layer

- ❑ When a neural network have one or more layer between input and output layers, they are called “**hidden layers**”
- ❑ Hidden layers have no connexion with the output world



Network Depth: From Shallow to Deep

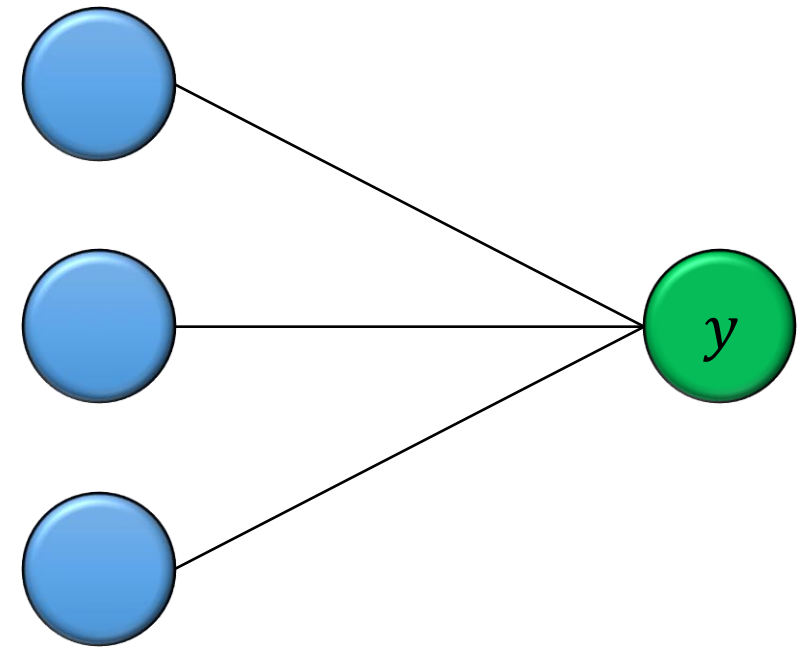
- ❑ Now that we understand basic Neural Network structure, we can explore the notion of deepness
- ❑ Neural Networks are called differently regarding their deepness



BrunelloN, CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0> via Wikimedia Commons

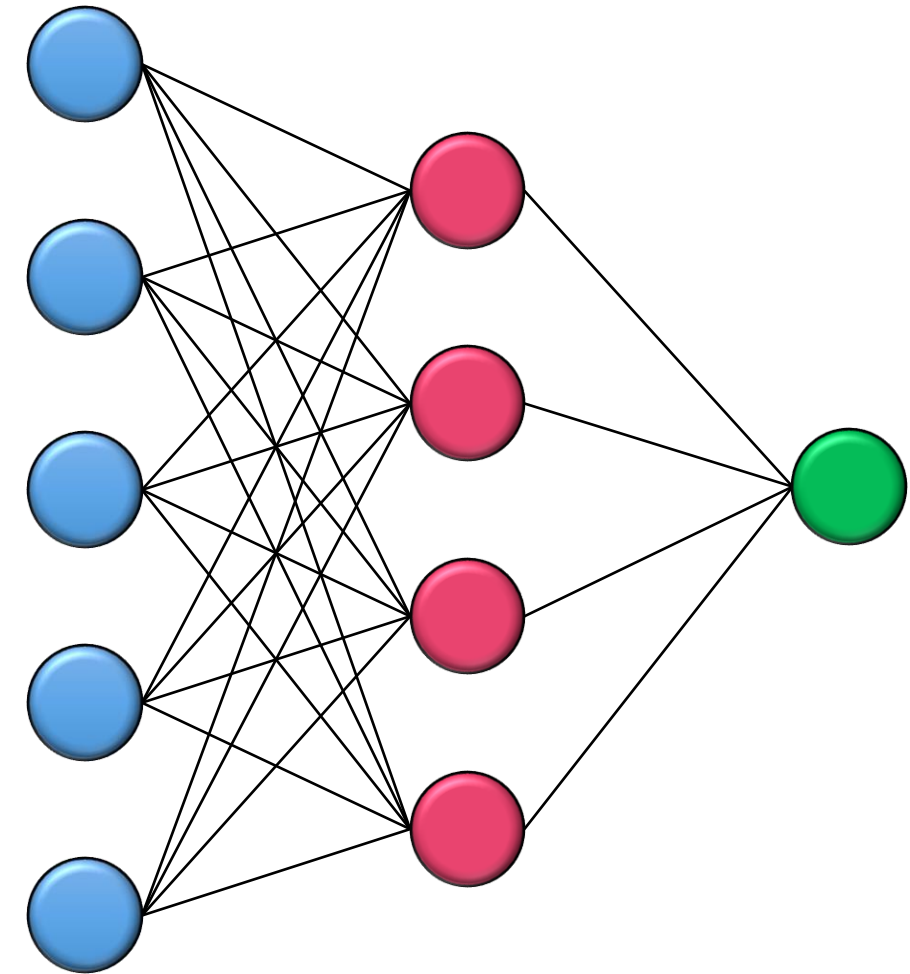
Single Layer Perceptron

- ❑ Is the simplest type of ANN
- ❑ Has **no hidden layer**
- ❑ Is composed of perceptron neurons.
So it can only handle linearly separable tasks (simple tasks)
- ❑ Mainly used for classification



Multilayer Perceptron

- ❑ Is used to solve complex problems
- ❑ Has **one or more hidden layer** between input and output
- ❑ Each layer is **Fully Connected** (“**Dense**”) to all neurons of previous layer
- ❑ The architecture is called **Feedforward Neural Network** (information flows in one direction, no loops)
- ❑ Is the foundation of modern neural networks



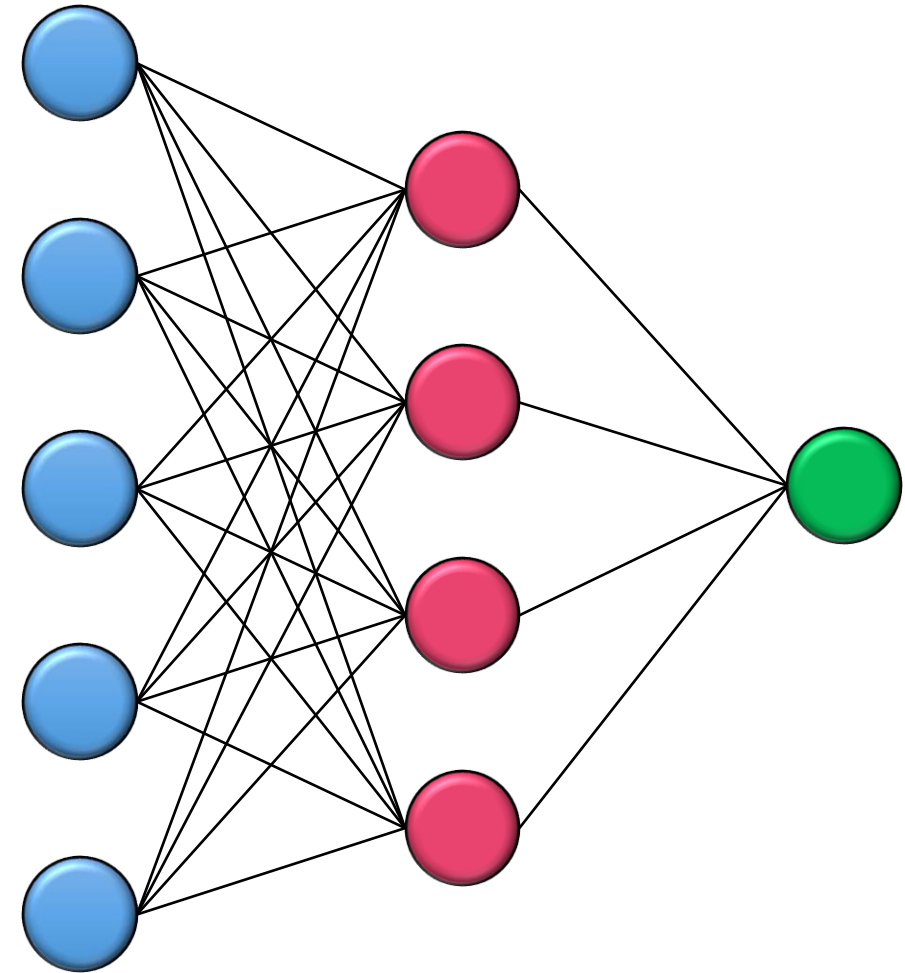
Multilayer Perceptron

- ❑ A MLP with **only one hidden** layer is called **Shallow Neural Network**, or **Non-Deep Neural Network**



Even though “perceptron” is in their name, multi-layer perceptrons (MLPs) are not made up of original perceptrons.

They can use **any type of non-linear activation function**



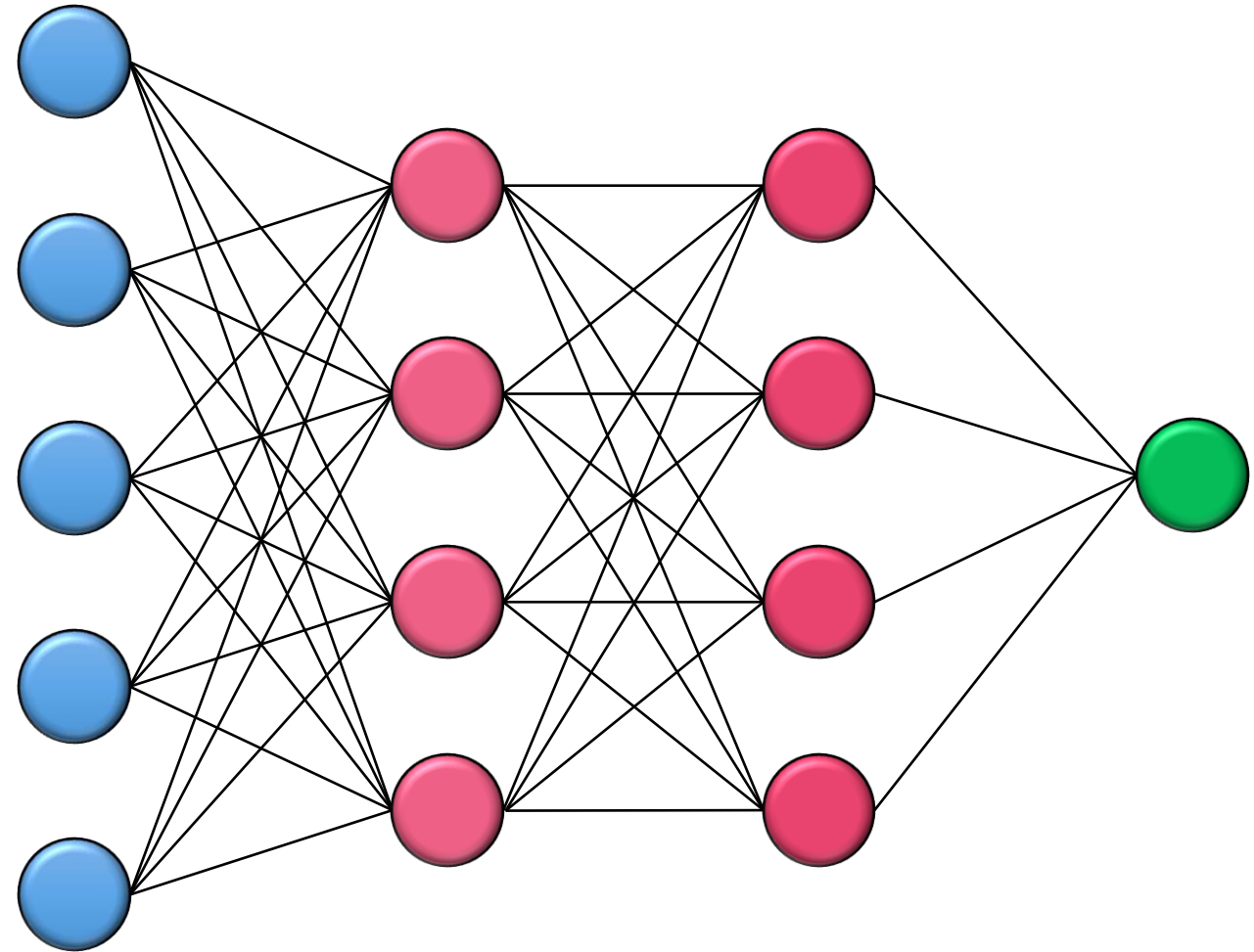
Multilayer Perceptron

- ❑ A MLP with **more than one hidden layer** is called **Deep Neural Network**

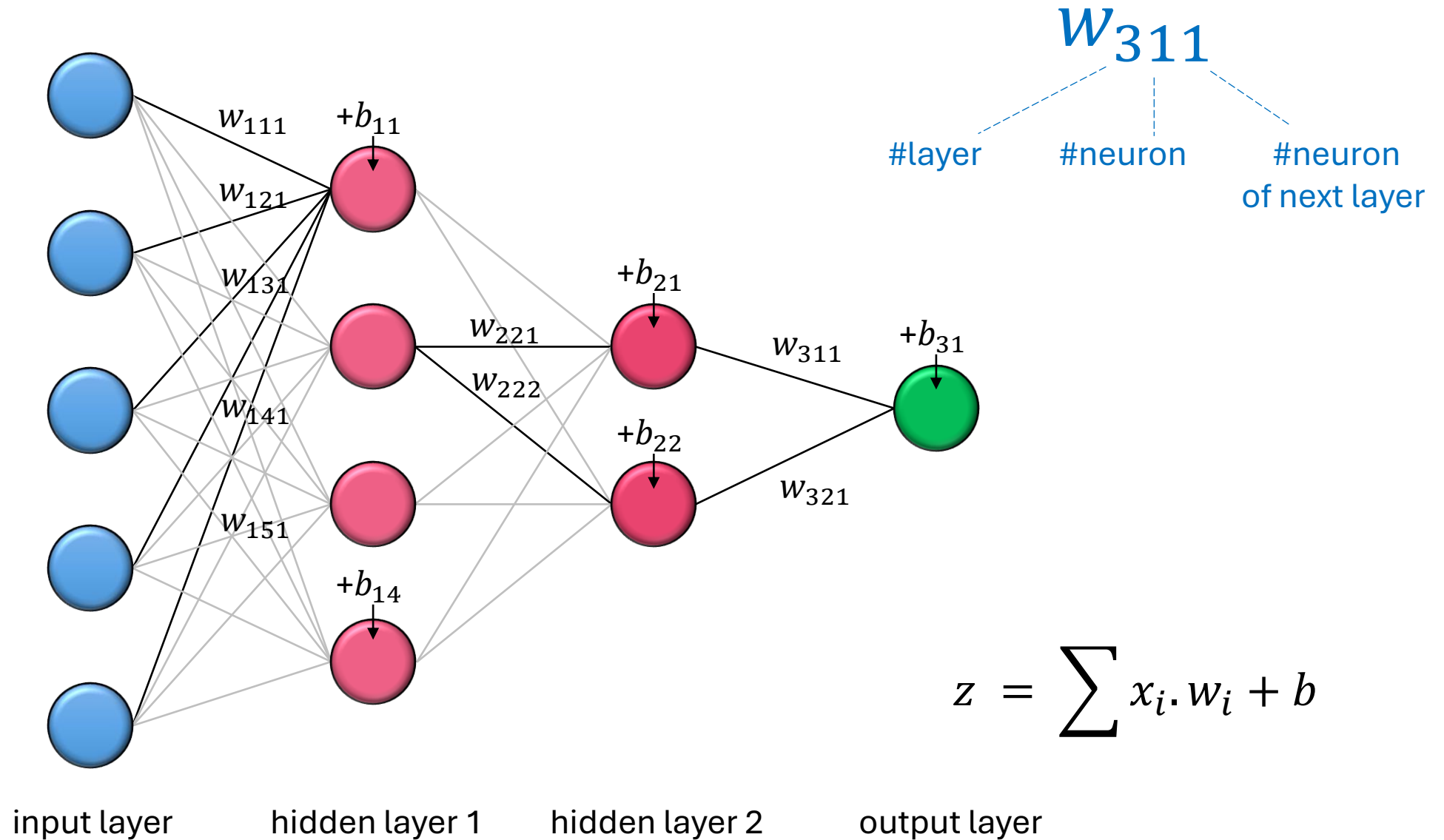


Even though “perceptron” is in their name, multi-layer perceptrons (MLPs) are not made up of original perceptrons.

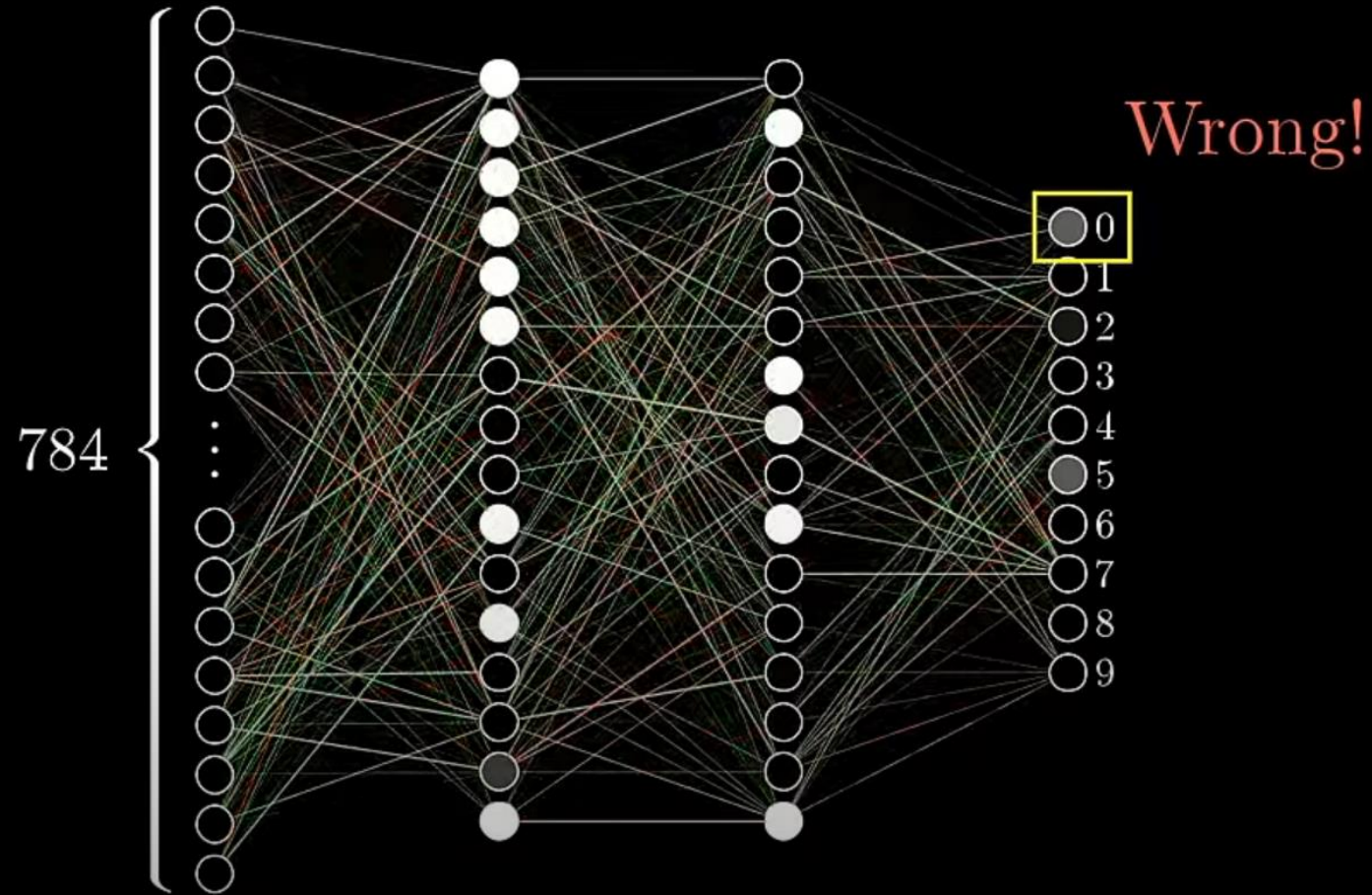
They can use **any type of non-linear activation function**



Weights and biases notation



What weights are used here?
What are they doing?

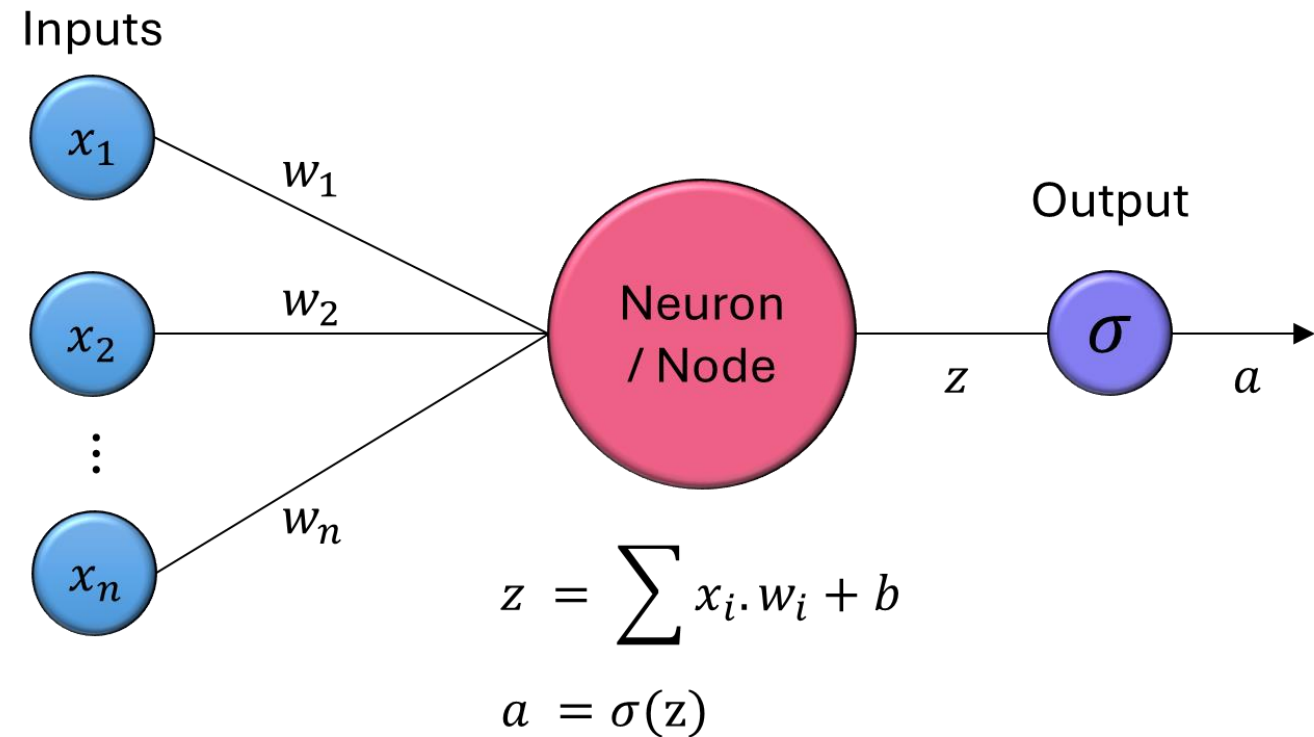


3 Blue One Brown: [What is A Neural Network](#)

Activation Functions

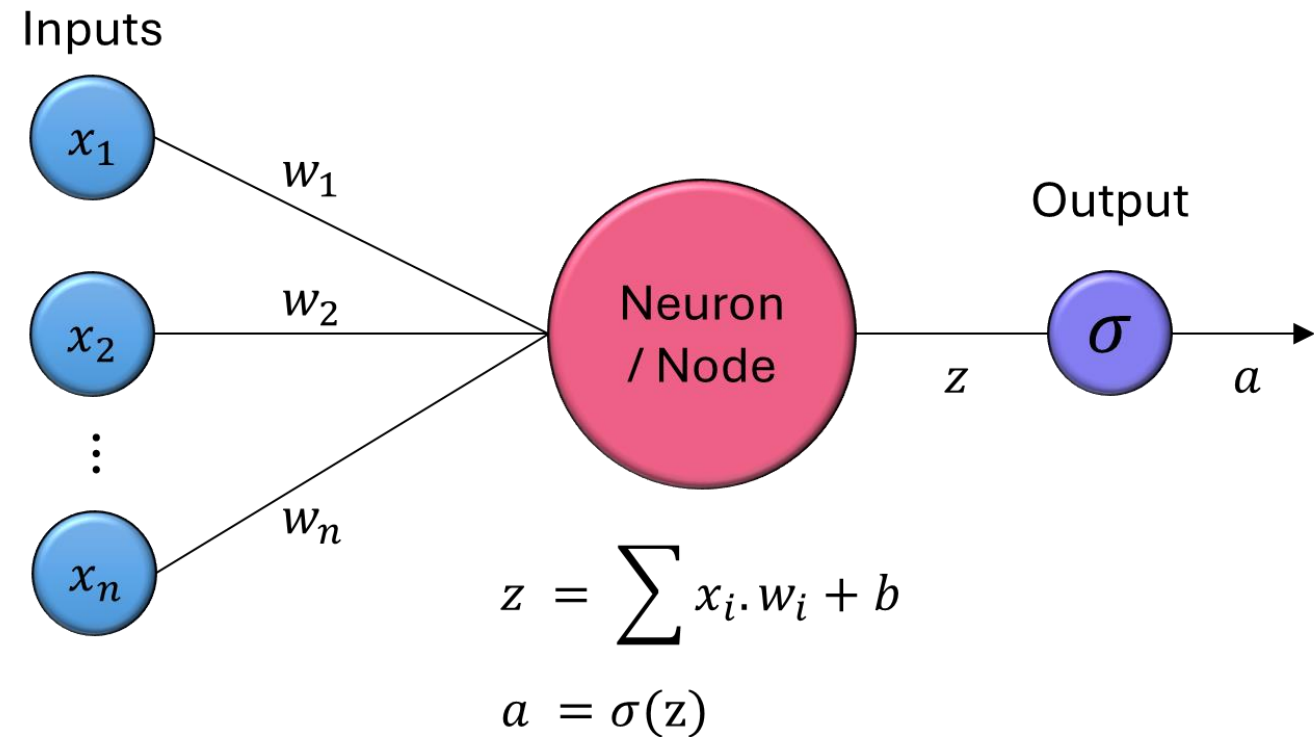
What are activation functions?

- ❑ Activation functions are **decision making units** of neural networks
- ❑ They are mathematical equations, attached to each neuron
 - They check the value that the neuron receives
 - They determine its output, whether the neuron should be activated or not



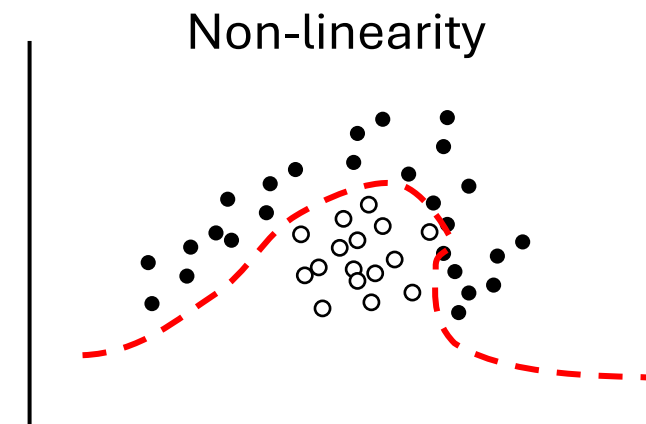
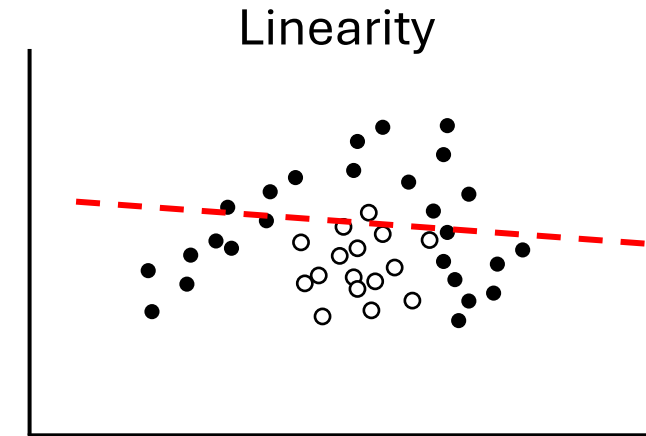
What are activation functions?

- ❑ They can also be used to normalize output of a neuron
 - range between 0 & 1
 - or range between -1 and 1
- ❑ They can thus be used to determine the output of a Neural Network (e.g. probability between 0 and 1)



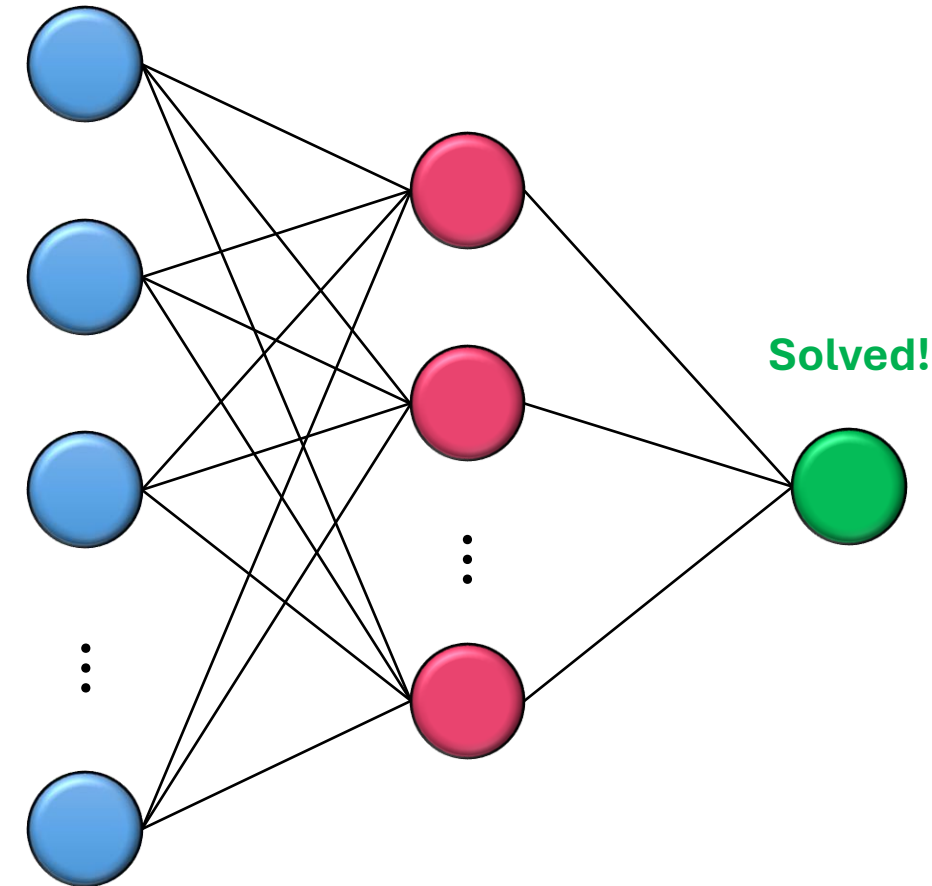
Non linearity

- ❑ **Activation functions are essential for NN**, because they introduce **non-linearity** into the model, allowing it to learn complex relationships in the data
- ❑ Without them, all layers would collapse into a single linear transformation
- ❑ Non-linear activation functions let the network combine features in flexible ways, enabling it to approximate highly complex functions and capture real-world patterns



Universal Approximation Theorem

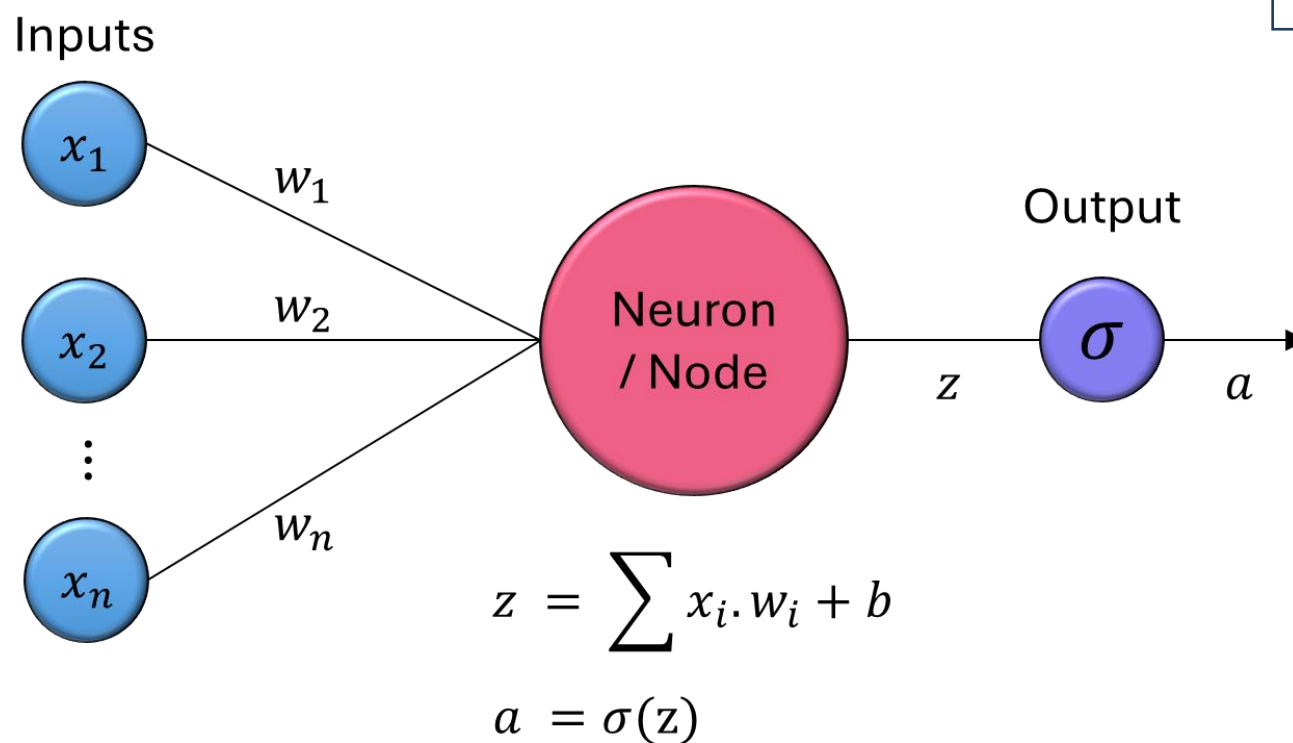
- ❑ The **Universal Approximation Theorem** (*Hornik et al, 1991*) claims that at least one neural network exists that can approximate any continuous function with arbitrary precision
- ❑ ... Even with a single hidden layer
- ❑ ... Thanks to any non-polynomial activation functions (generalization by *Leshno, 1993*)



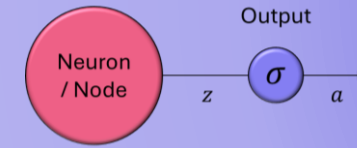
- ❑ Let's review common activation functions
- ❑ Remember that z is their input, and a is their output



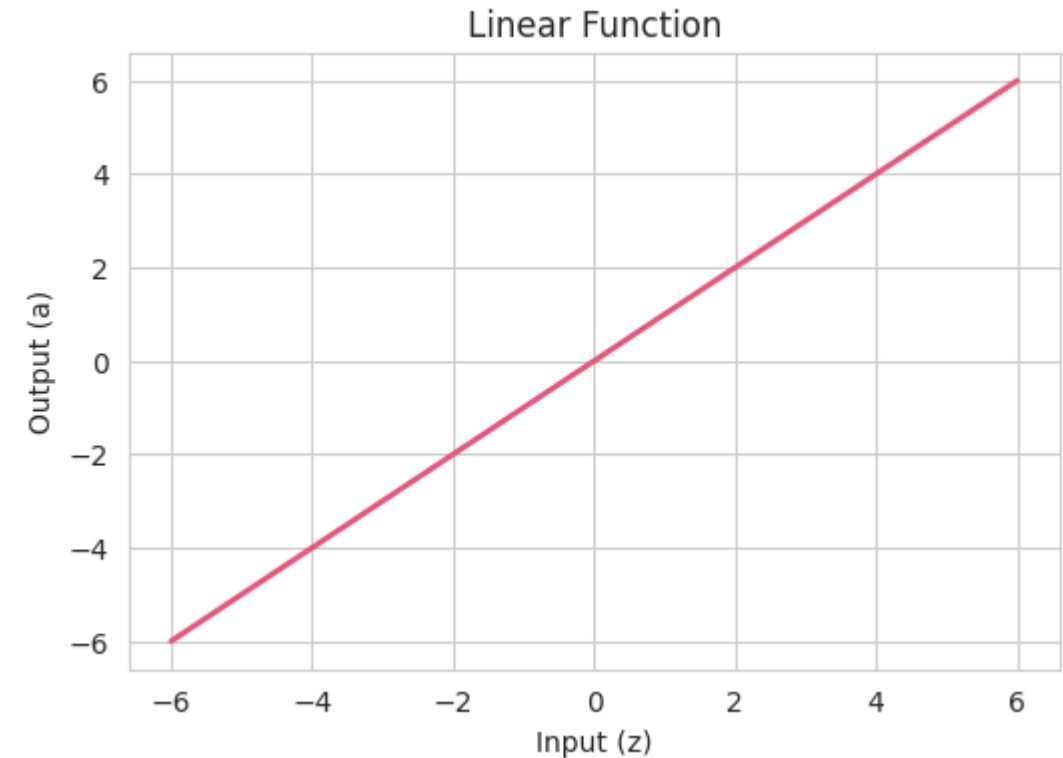
This cup points out
most important
functions to know



Linear Activation

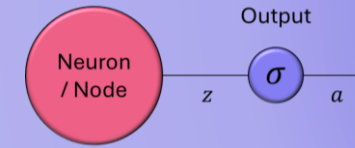


- ❑ Is also called “no activation”, or “identity function”
- ❑ Output = Input
- ❑ Practically speaking, “Linear activation” means there is no activation function.



 **Does not introduce non-linearity!**

Binary Step Function

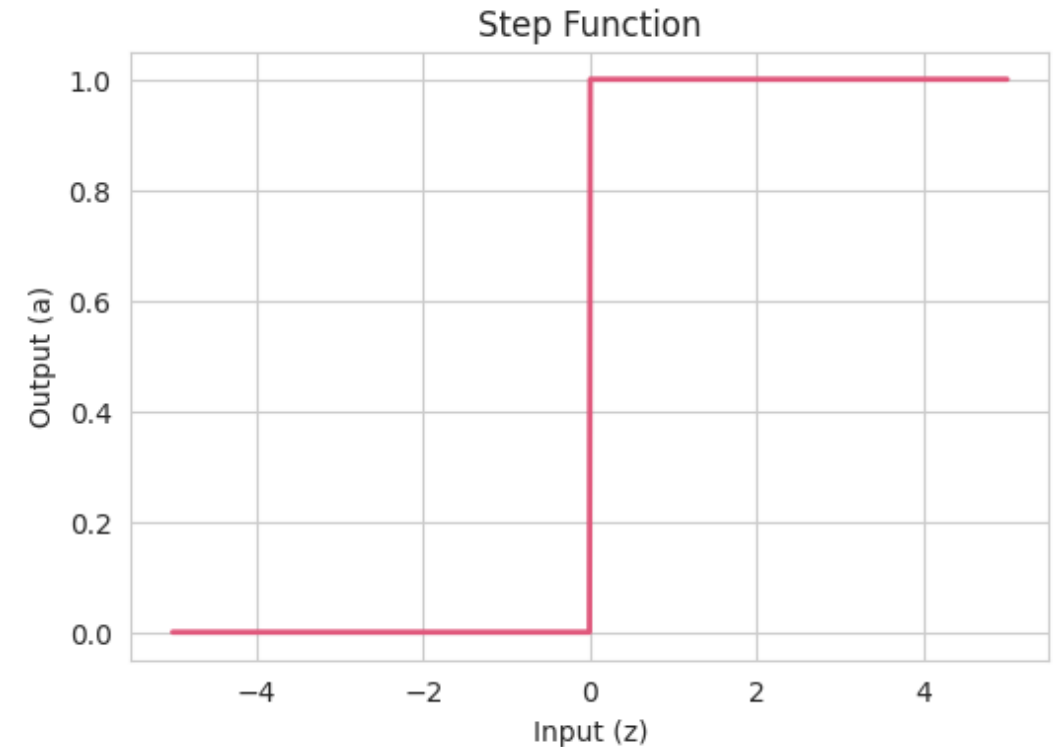


- ❑ Is the activation function of the original Perceptron (1957)
- ❑ Is Threshold-based

$$a = \begin{cases} 1 & \text{if } z \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

θ : “threshold” or “bias”

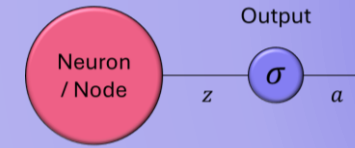
- ❑ Neuron is activated (“fired”), or deactivated



 **Does not introduce non-linearity!**

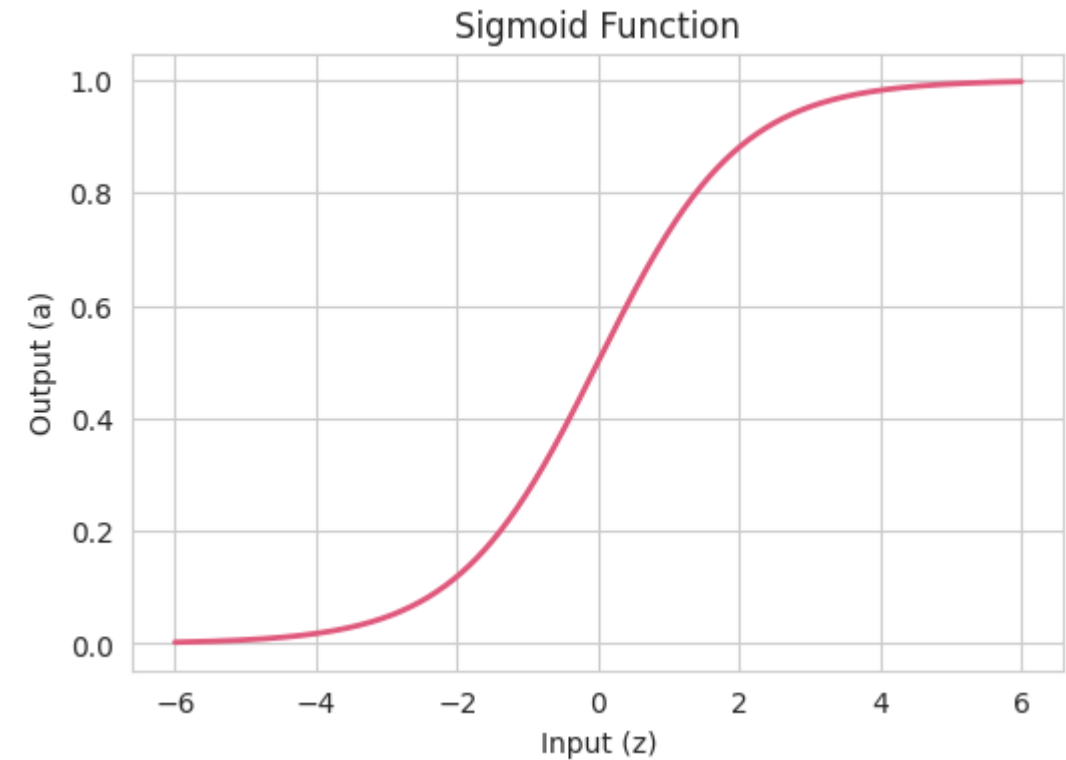


Sigmoid (or Logistic) Function



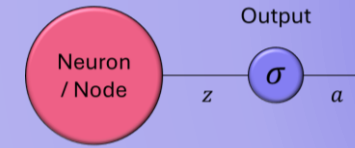
- ❑ Output values are between 0 and 1
- ❑ they can be interpreted as probabilities

$$S(z) = \frac{1}{1 + e^{-z}}$$





Sigmoid (or Logistic) Function



- ❑ Output values are between 0 and 1
- ❑ they can be interpreted as probabilities

$$S(z) = \frac{1}{1 + e^{-z}}$$



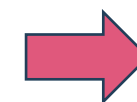
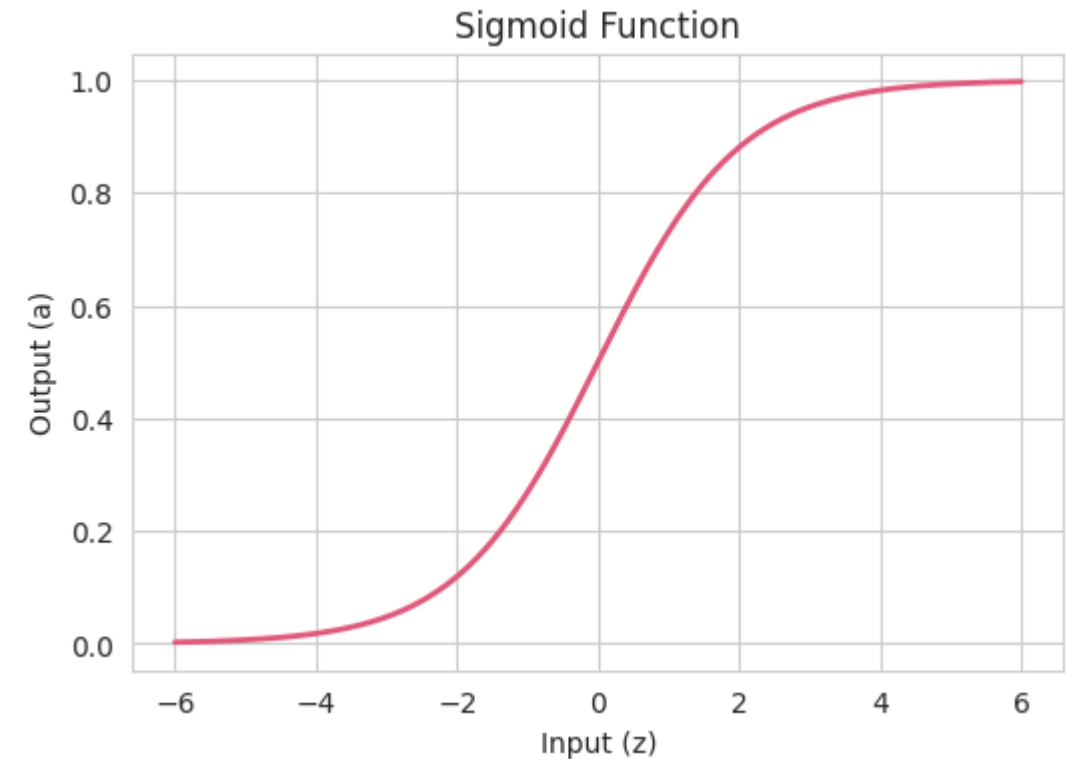
Adds non linearity



Computationally expensive

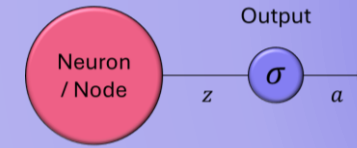


Problem of vanishing gradients (to be seen later)



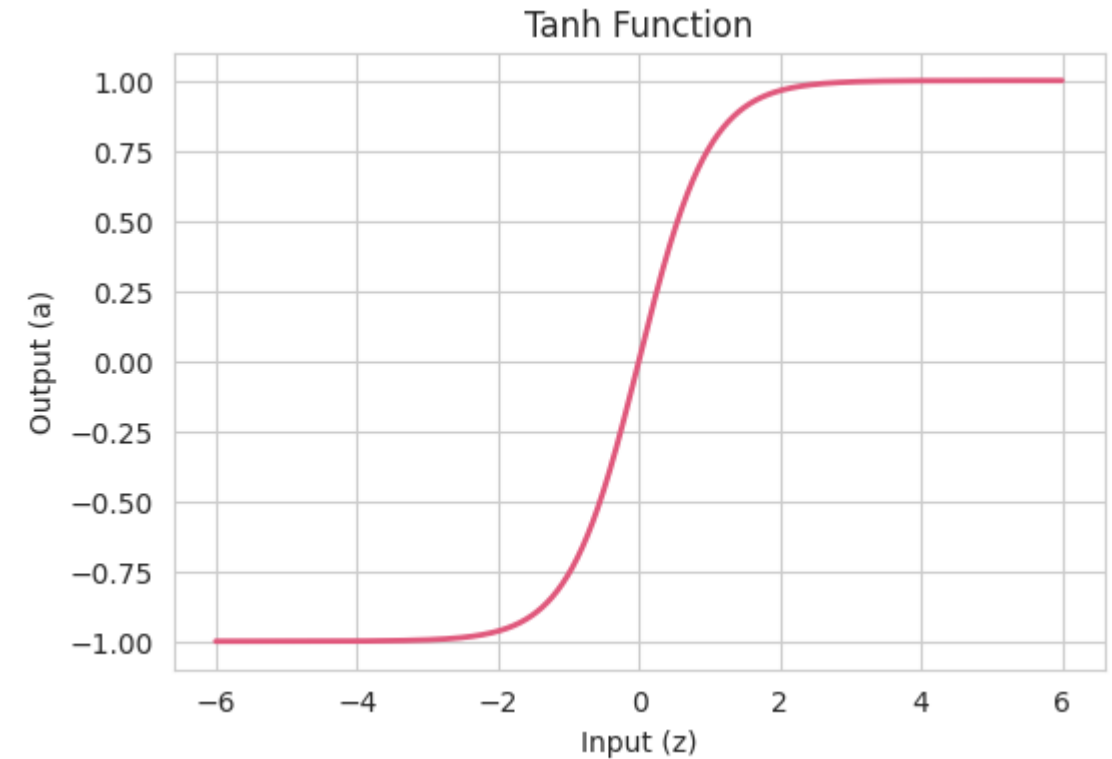
Mostly used for ANN output layer

Hyperbolic Tangent Function (TanH)

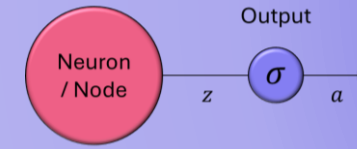


- ❑ Output values are between -1 and 1
- ❑ Zero centered

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



Hyperbolic Tangent Function (TanH)



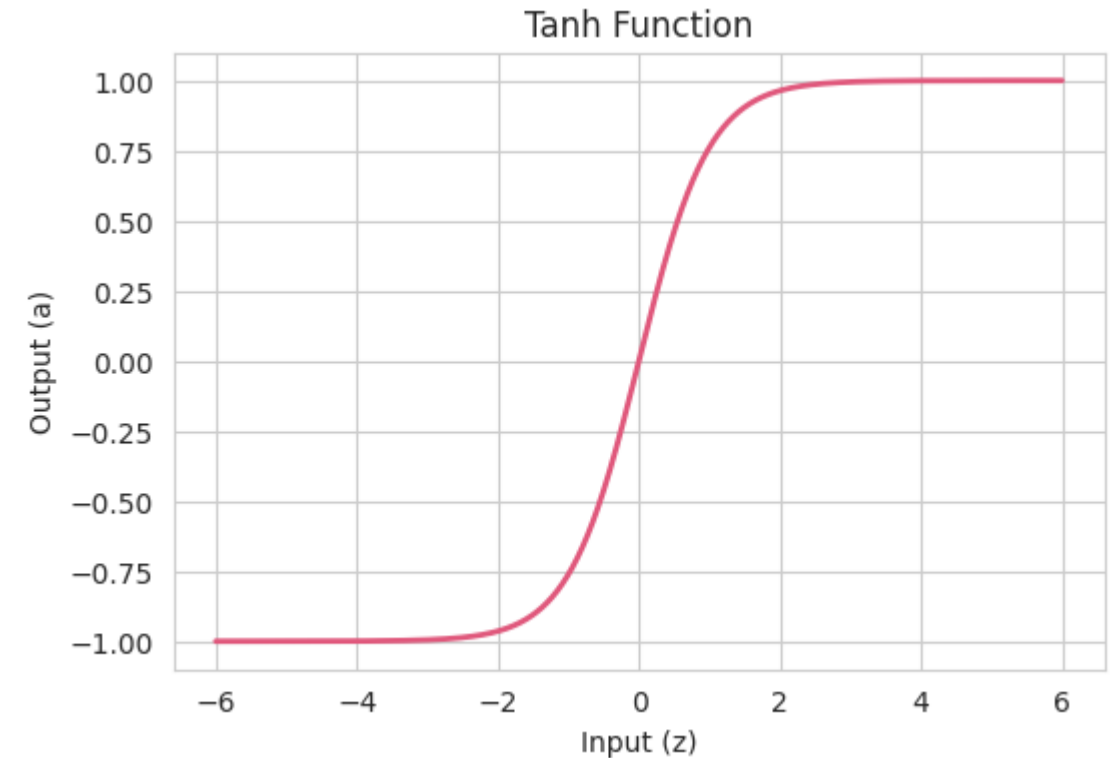
- ❑ Output values are between -1 and 1
- ❑ Zero centered

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

👍 Adds non linearity

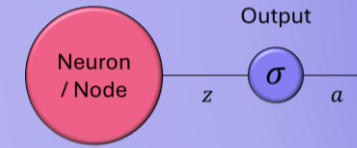
👎 Computationally expensive

👎 Problem of vanishing gradients (to be seen later)



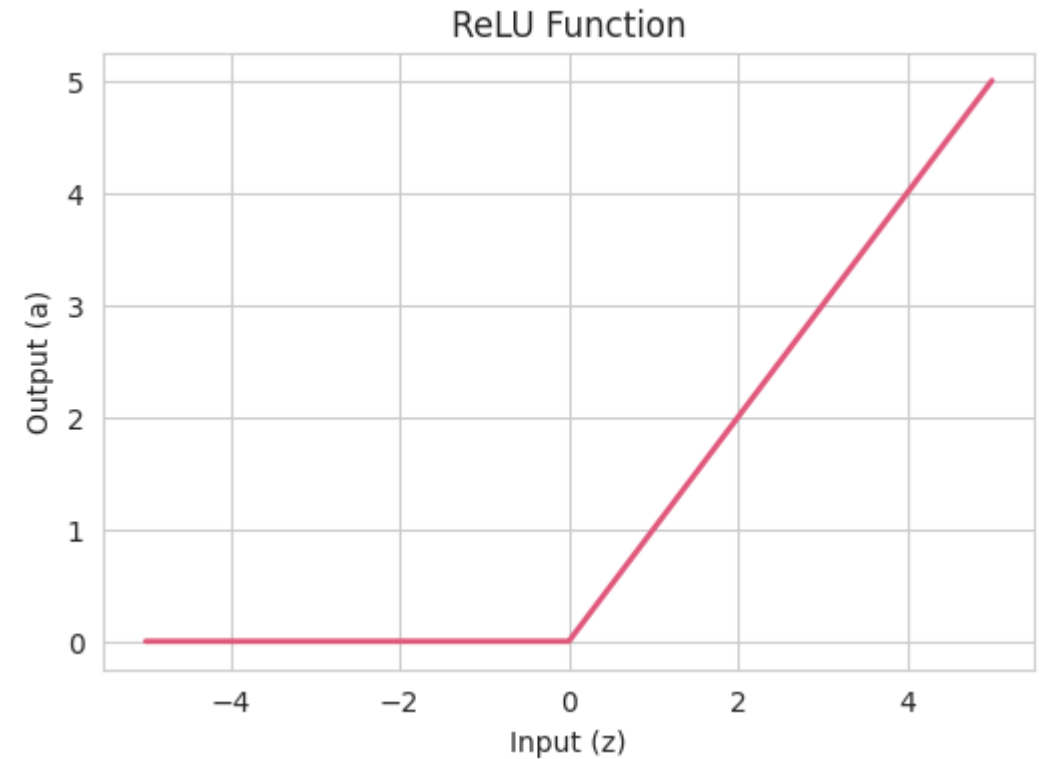


Rectified Linear Unit (ReLU)



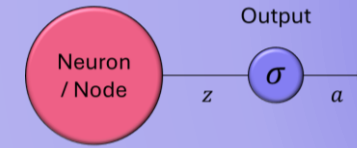
- ❑ Returns 0 if input < 0
- ❑ Returns identical value if input > 0

$$\text{ReLU}(z) = \max(0, z)$$





Rectified Linear Unit (ReLU)



- ❑ Returns 0 if input < 0
- ❑ Returns identical value if input > 0

$$\text{ReLU}(z) = \max(0, z)$$



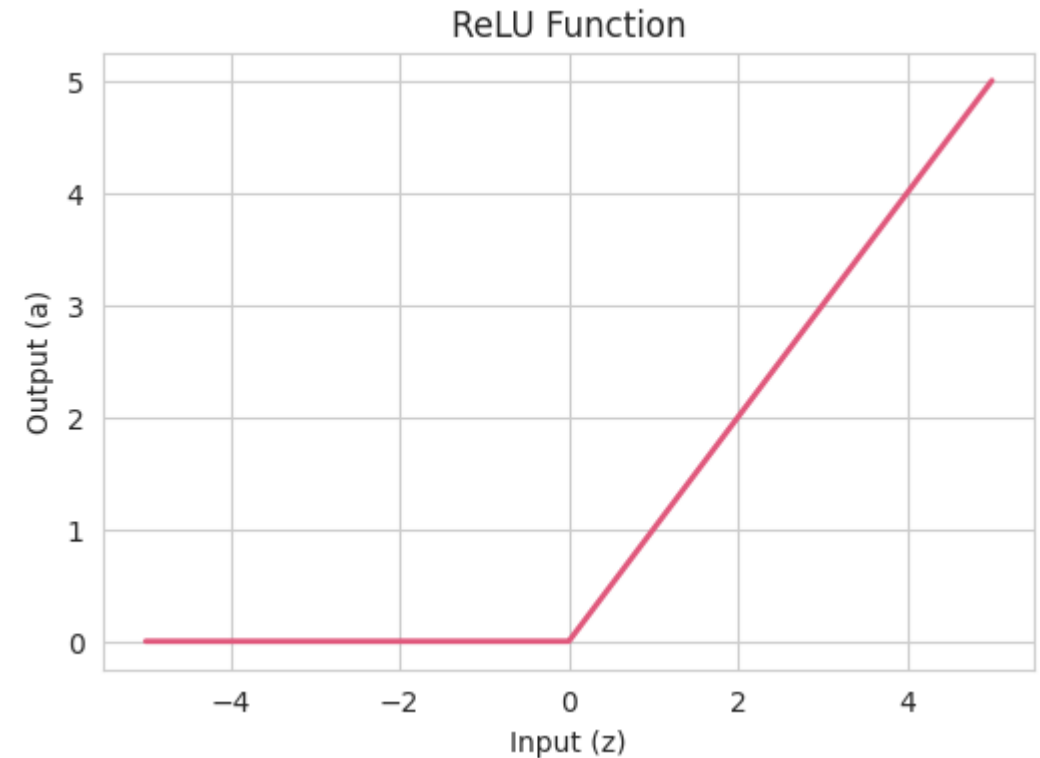
Adds non linearity



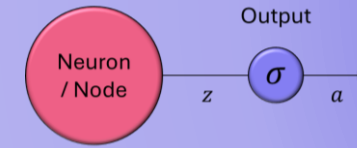
Computationally efficient: allows the network to converge fast



Dying ReLU problem: when the input is ≤ 0 , the network cannot learn

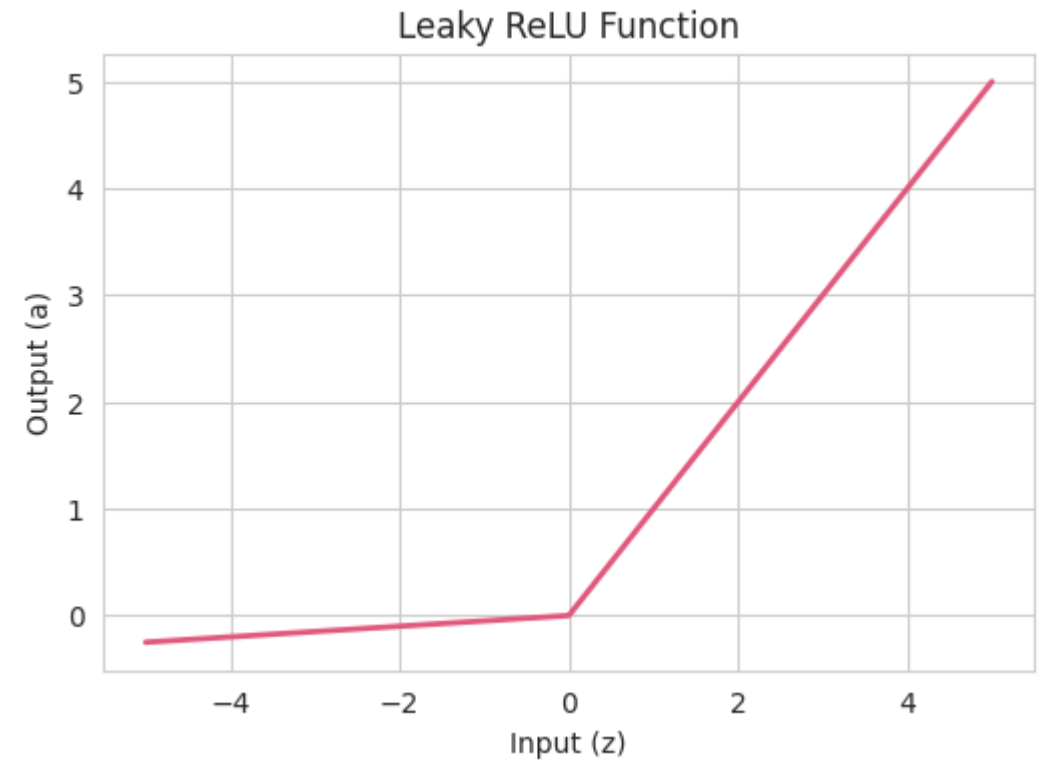


Leaky ReLU

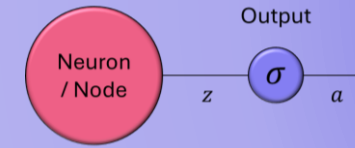


- ❑ Prevents the dying ReLU problem by adding a small positive slope on the negative side, so learning can be done

$$\text{Leaky_ReLU}(z) = \max(\alpha * z, z)$$



Leaky ReLU



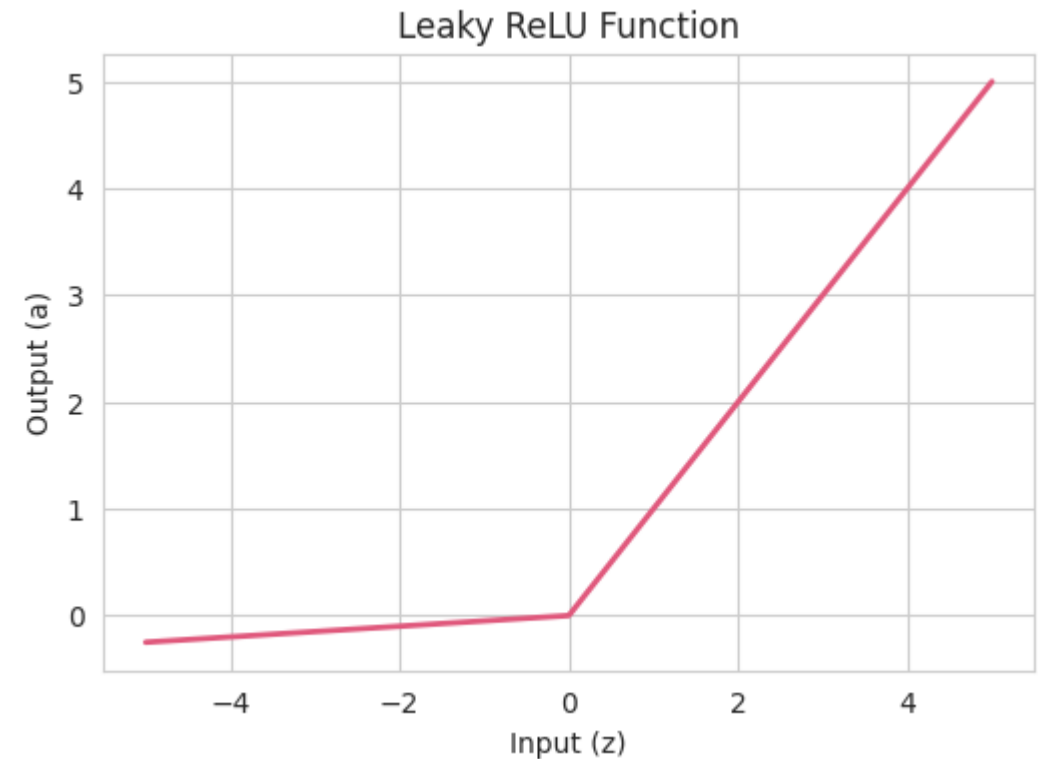
- ❑ Prevents the dying ReLU problem by adding a small positive slope on the negative side, so learning can be done

$$\text{Leaky_ReLU}(z) = \max(\alpha * z, z)$$

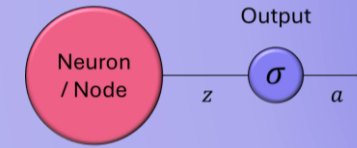
👍 Adds non linearity

👍 Computationally efficient

👎 Since the slope for negatives is arbitrary (α), the treatment of negative inputs can be inconsistent and may depend heavily on this value

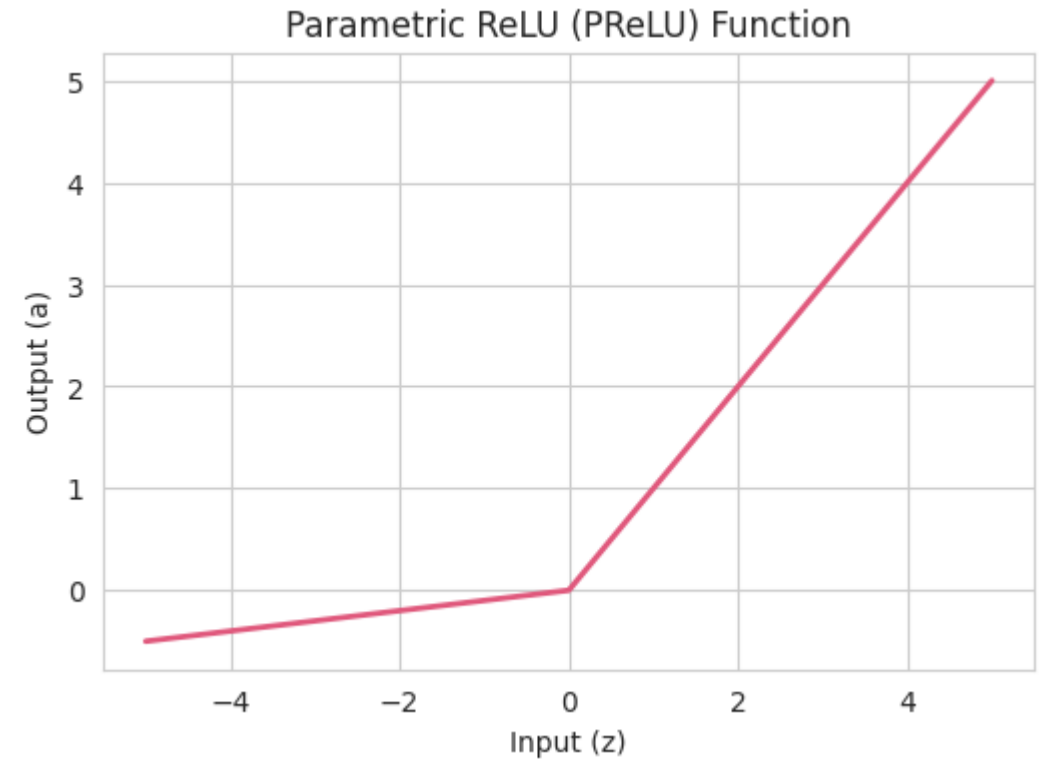


Parametric ReLU (PReLU)

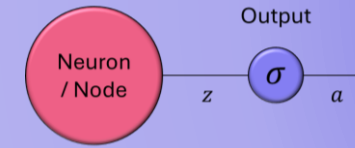


- ❑ Its leakage coefficient (slope α) is a learned parameter
- ❑ Prevents the dying ReLU problem

$$PReLU(z) = \begin{cases} z & \text{if } z \geq 0 \\ \alpha z & \text{otherwise} \end{cases}$$



Parametric ReLU (PReLU)



- ❑ Its leakage coefficient (slope α) is a learned parameter
- ❑ Prevents the dying ReLU problem

$$PReLU(z) = \begin{cases} z & \text{if } z \geq 0 \\ \alpha z & \text{otherwise} \end{cases}$$



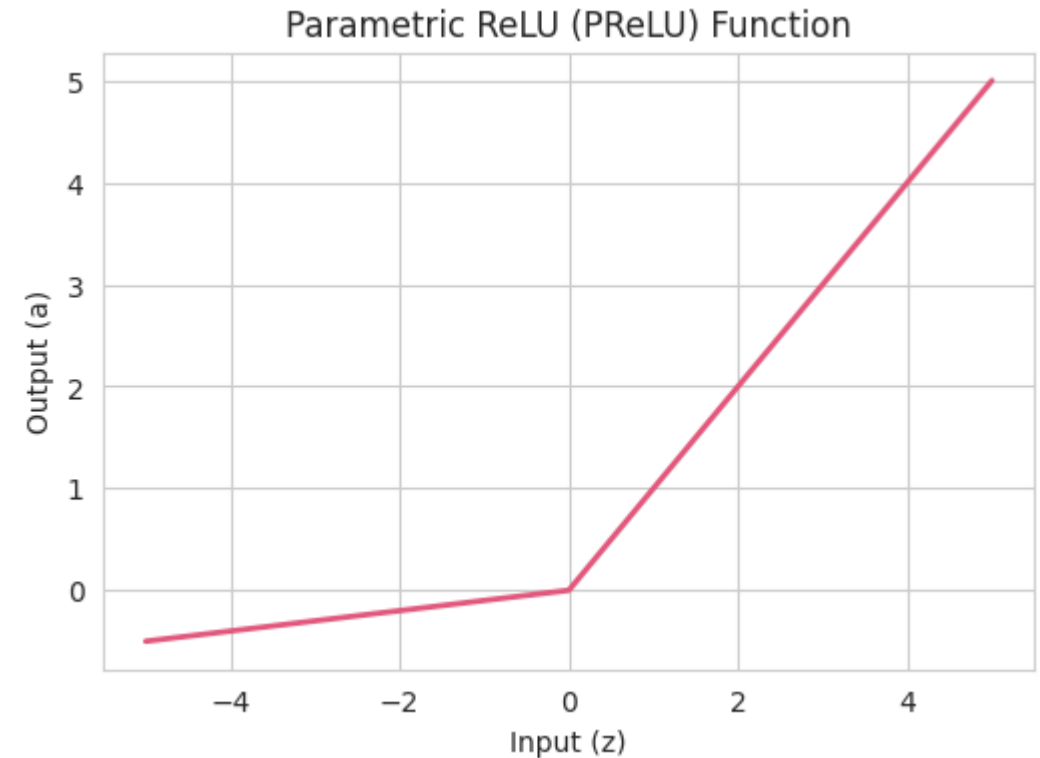
Adds non linearity



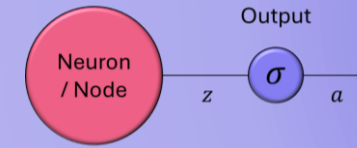
Computationally efficient



Add extra parameters to learn, for each neuron

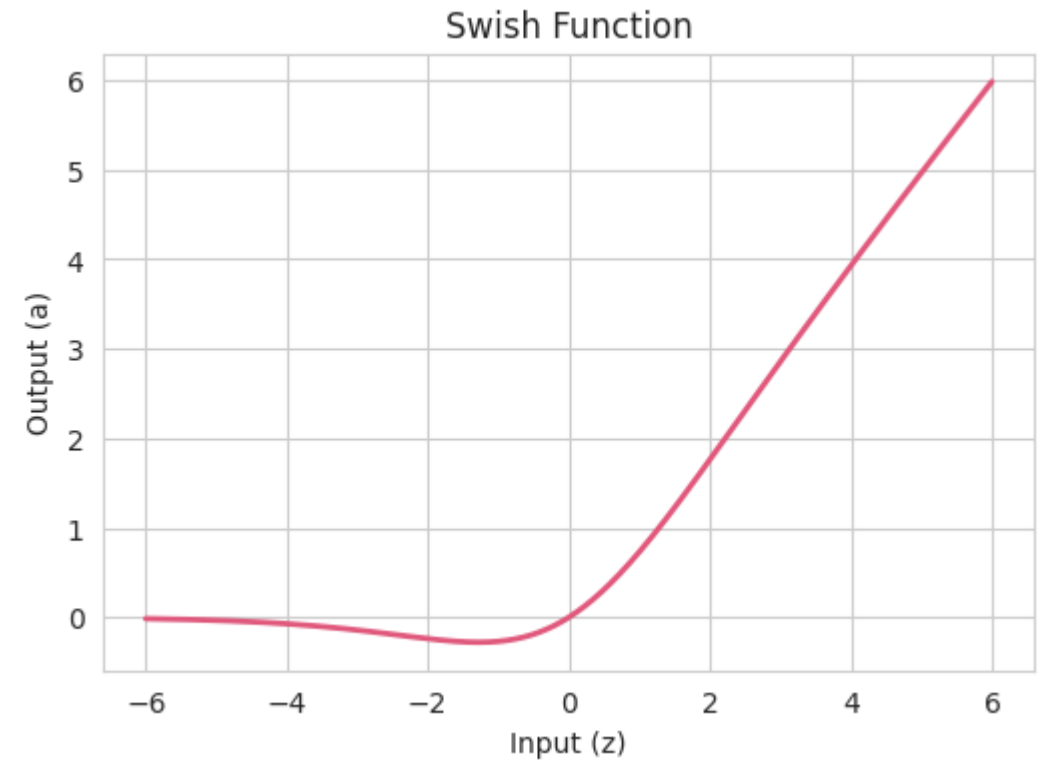


Swish Activation Function

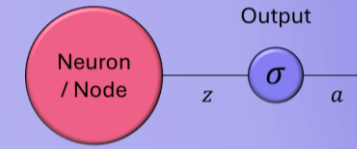


- ❑ Often outperforms ReLU for deep networks

$$\text{Swish}(z) = \frac{z}{1 + e^{-z}}$$



Swish Activation Function



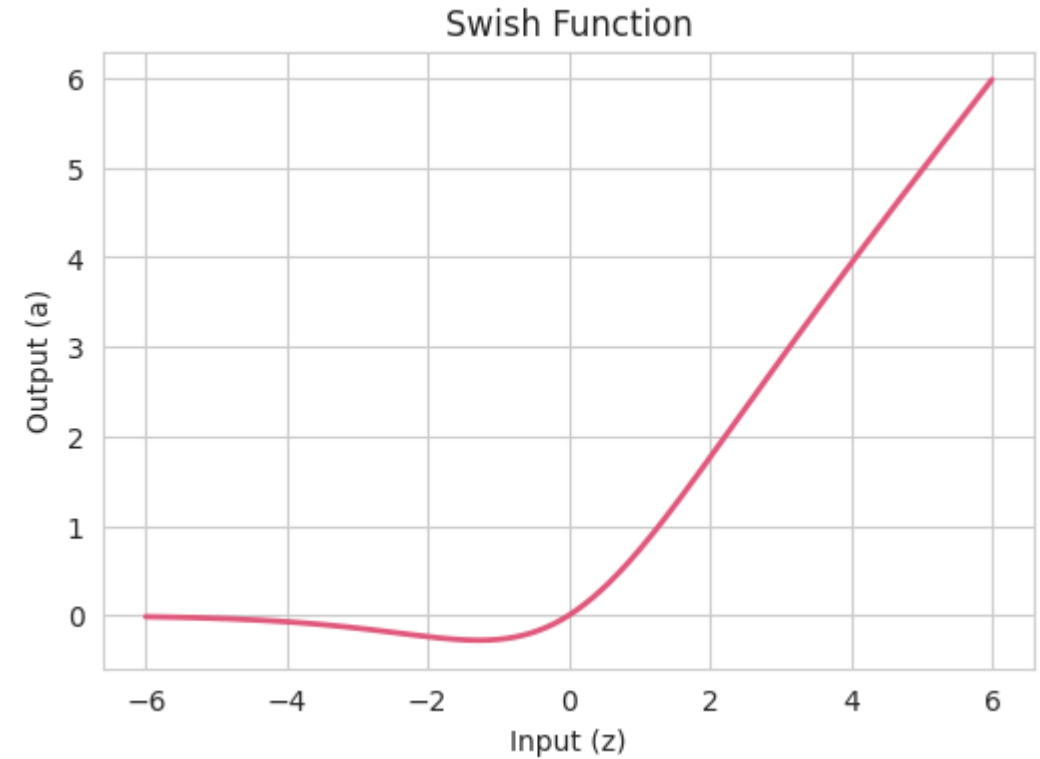
- ❑ Often outperforms ReLU for deep networks

$$\text{Swish}(z) = \frac{z}{1 + e^{-z}}$$

👍 Adds non linearity

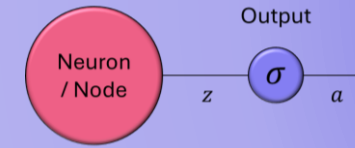
👍 Avoids dying ReLU problem

👎 Computationally expensive

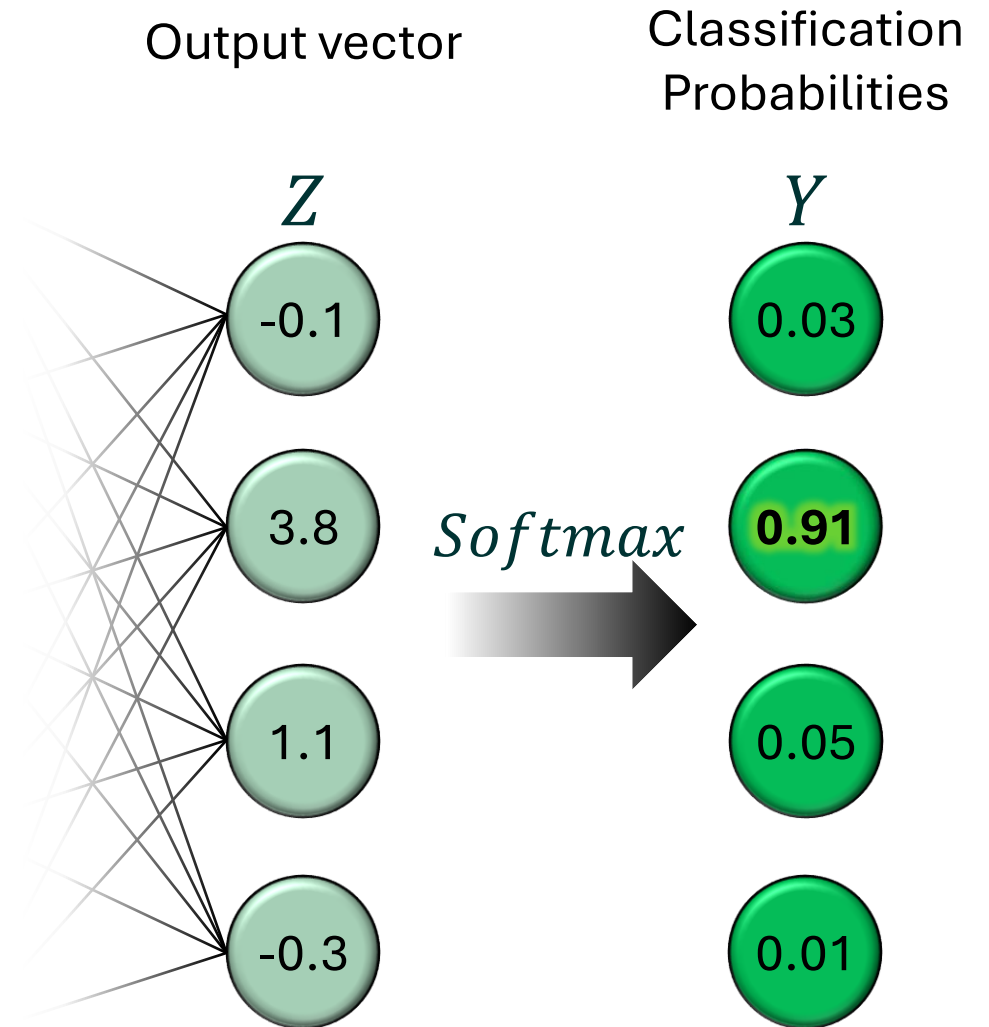
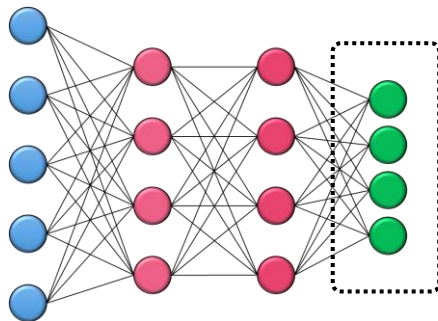




Softmax Activation Function

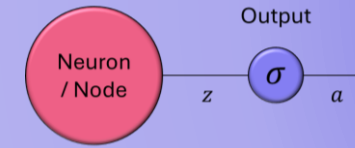


- ❑ Is **used in the output layer** of multiclass classification models (to be seen later)
- ❑ Converts raw model outputs (logits) into probabilities that sum to 1
- ❑ Makes the model's outputs humanly interpretable as probabilities

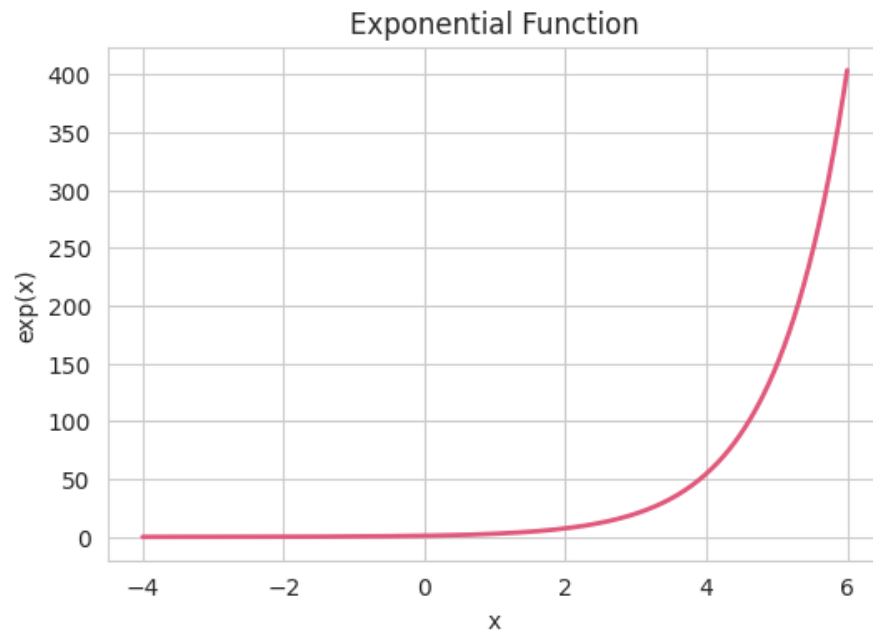




Softmax Activation Function



- ❑ Uses exponential function to make high values even higher
 - It makes sure that the probability of the most probable class stands out



Exponentiates each logit to ensure positive values

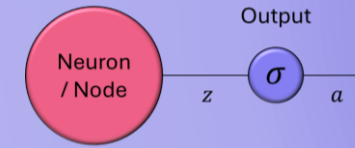
$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Denominator normalises so that all probabilities sum to 1

$$\sum_{i=1}^K Softmax(z_i) = 1$$



Softmax Activation Function



$$\text{Norm}(2.0) = \frac{2.0}{2.0 + 3.1 + 1.0} = 0.33$$

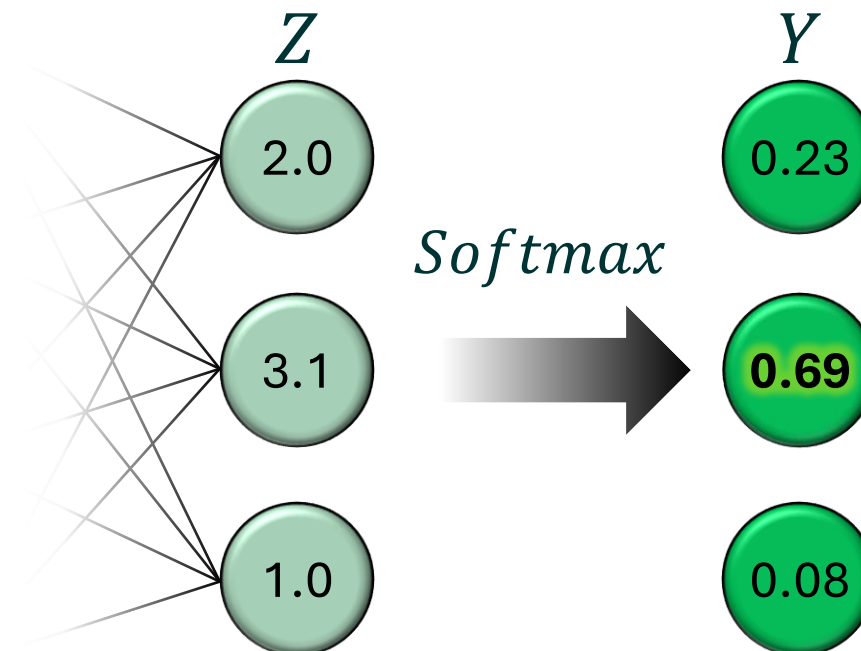
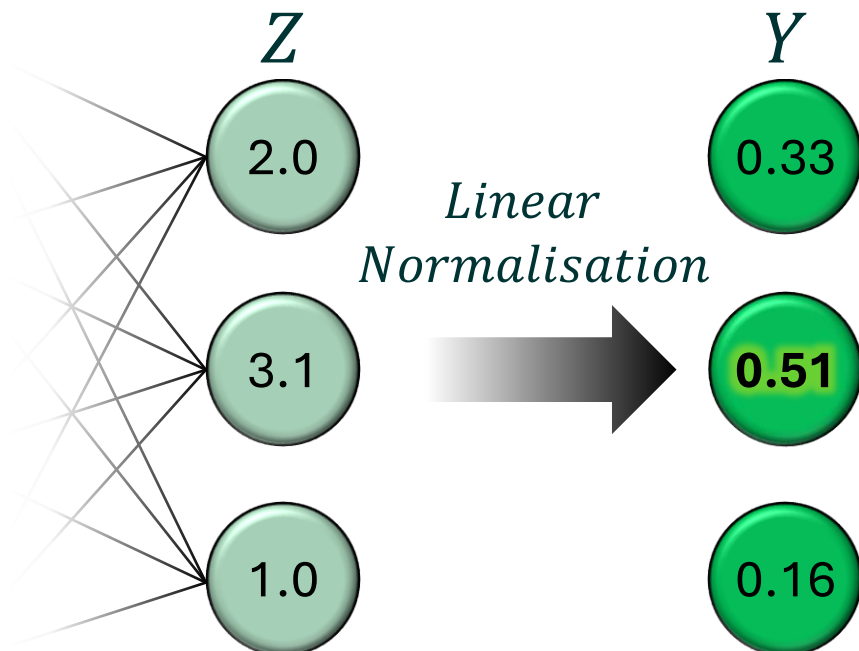
$$\text{Norm}(3.1) = \frac{3.1}{2.0 + 3.1 + 1.0} = 0.51$$

$$\text{Norm}(1.0) = \frac{1.0}{2.0 + 3.1 + 1.0} = 0.16$$

$$\text{Softmax}(2.0) = \frac{e^{2.0}}{e^{2.0} + e^{3.1} + e^{1.0}} = \frac{7.39}{32.30} = 0.23$$

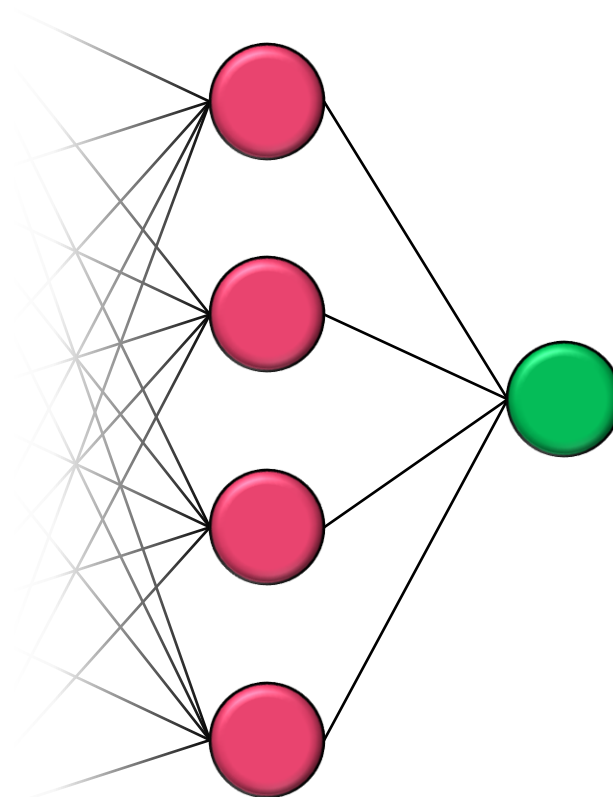
$$\text{Softmax}(3.1) = \frac{e^{3.1}}{e^{2.0} + e^{3.1} + e^{1.0}} = \frac{22.20}{32.30} = 0.69$$

$$\text{Softmax}(1.0) = \frac{e^{1.0}}{e^{2.0} + e^{3.1} + e^{1.0}} = \frac{2.72}{32.30} = 0.08$$



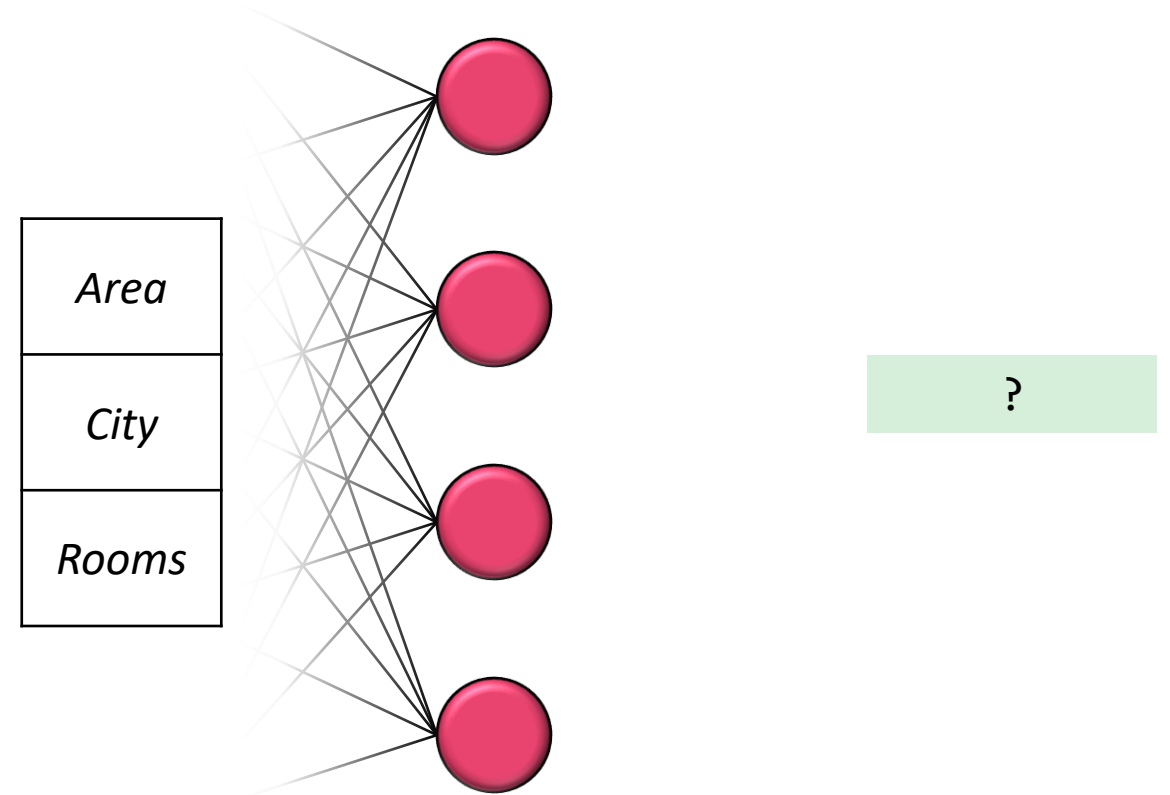
Types of ANN Outputs

- ❑ Once their fundamental elements are known, it is possible to explore all kinds of tasks that neural networks can perform
- ❑ Most of a neural network's architecture is dedicated to extracting and transforming features from the input data
- ❑ However, the task itself (regression, classification, etc.) is determined only by the output layer: its size and activation function



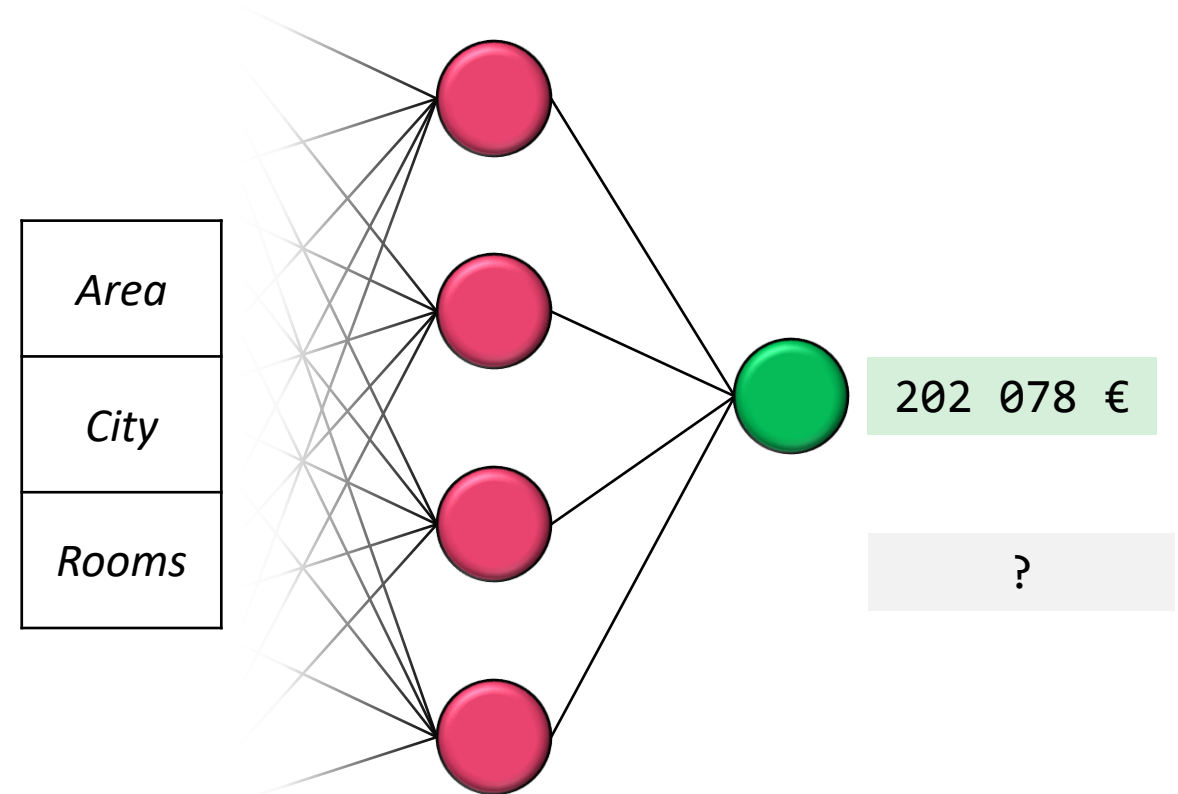
Regression

- ❑ Goal
 - **Predict a continuous numerical value**
 - e.g., house price, temperature, distance, duration in seconds...
- ❑ Output value
 - ?
- ❑ Output layer
 - **? neurons**



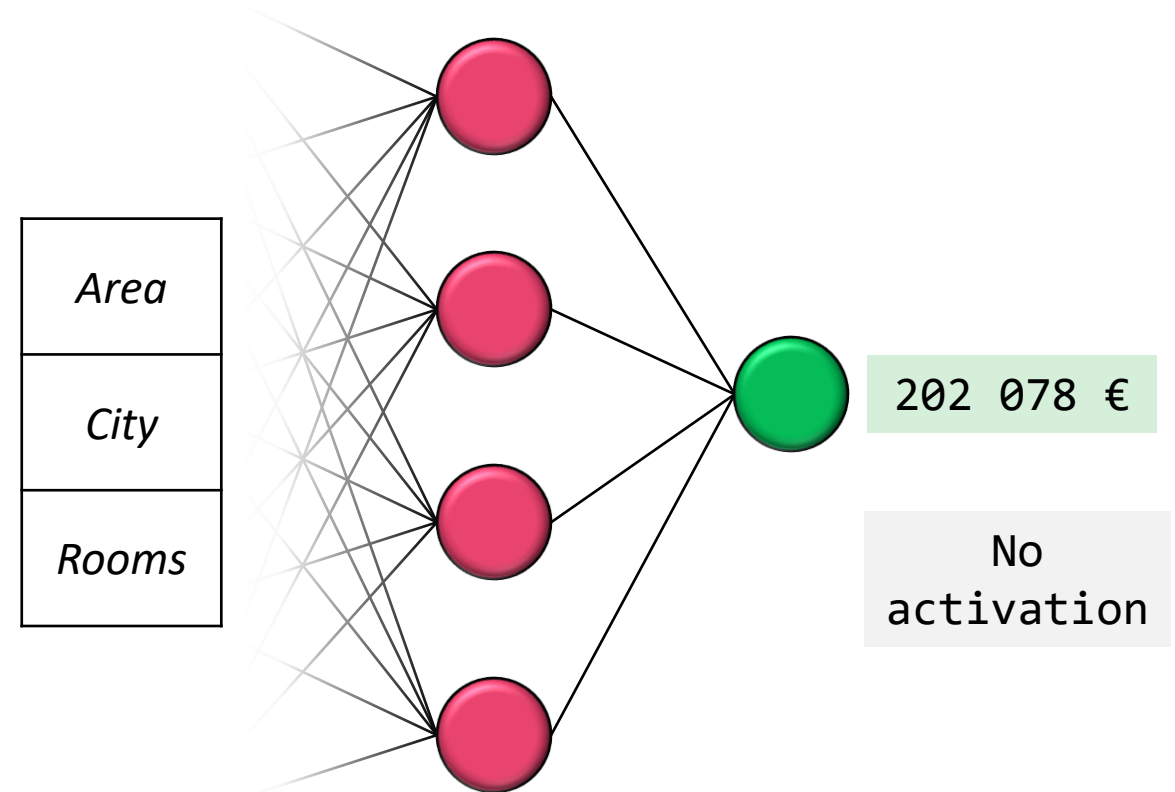
Regression

- ❑ Goal
 - **Predict a continuous numerical value**
 - e.g., house price, temperature, distance, duration in seconds...
- ❑ Output value
 - Any real number $\in [-\infty; +\infty]$
- ❑ Output layer
 - **1 neuron**
 - Activation: ?



Regression

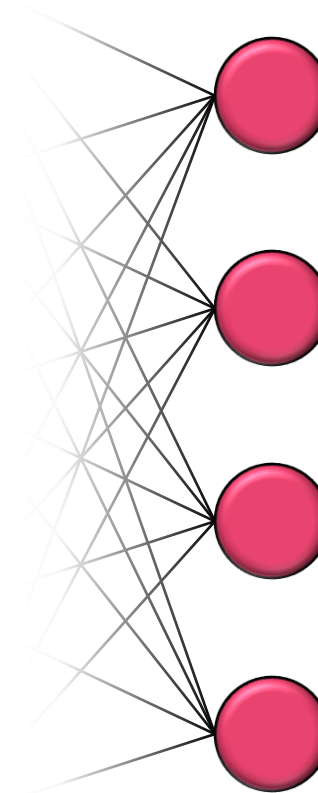
- ❑ Goal
 - **Predict a continuous numerical value**
 - e.g., house price, temperature, distance, duration in seconds...
- ❑ Output value
 - Any real number $\in [-\infty; +\infty]$
- ❑ Output layer
 - **1 neuron**
 - Activation: **No Activation**



Binary Classification

- Goal
 - **Predict one of two classes.** Class #1 is in general the one to “detect”
 - e.g., “*not spam*” / “*spam*”, “*negative behavior*” / “*positive*”, “*Authentic*” / “*Fake*”...
- Output value
 - ?
- Output layer
 - ? neurons

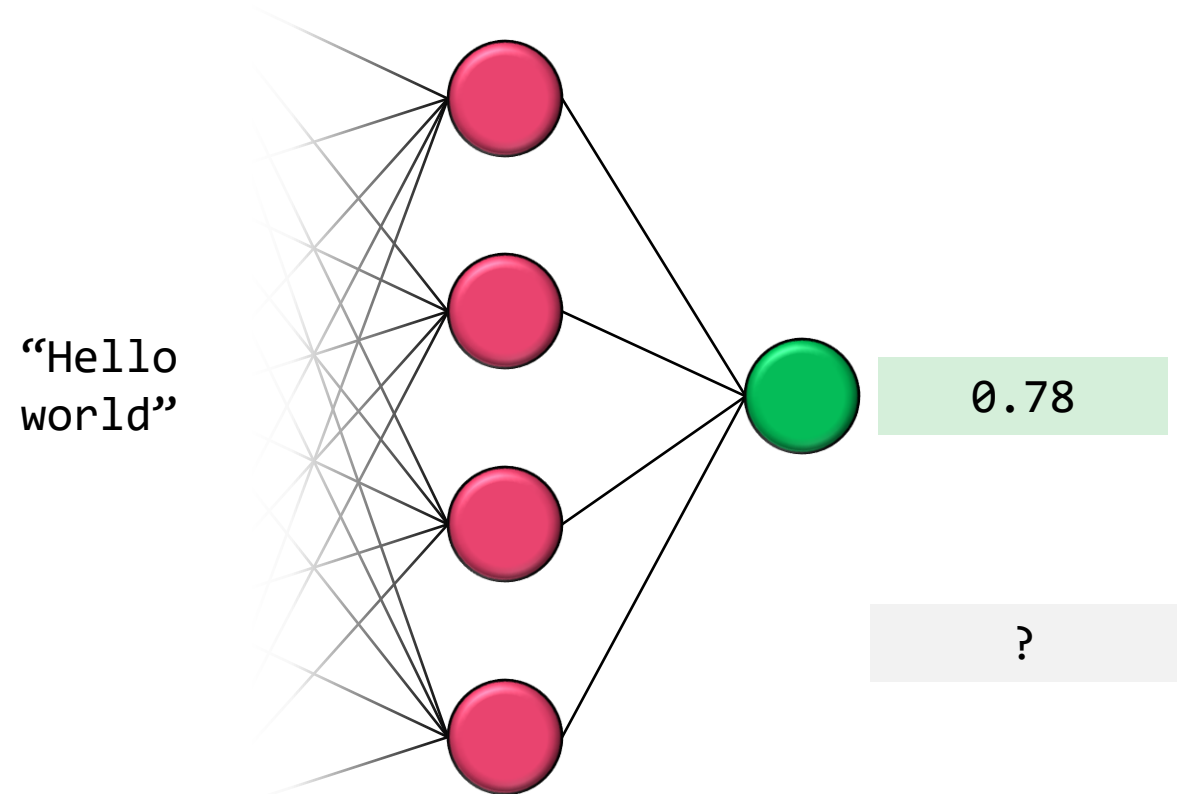
“Hello
world”



?

Binary Classification

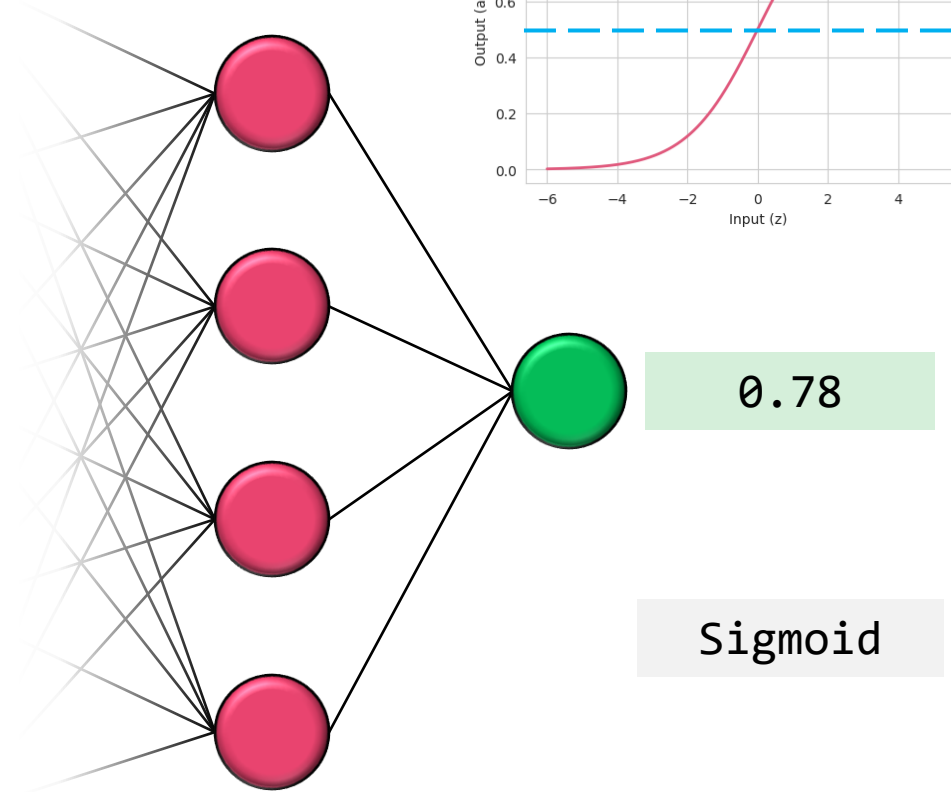
- ❑ Goal
 - **Predict one of two classes.** Class #1 is in general the one to “detect”
 - e.g., “not spam” / “spam”, “negative behavior” / “positive”, “Authentic” / “Fake”...
- ❑ Output value
 - Probability $\in [0; 1]$
- ❑ Output layer
 - **1 neuron**
 - Activation: ?



Binary Classification

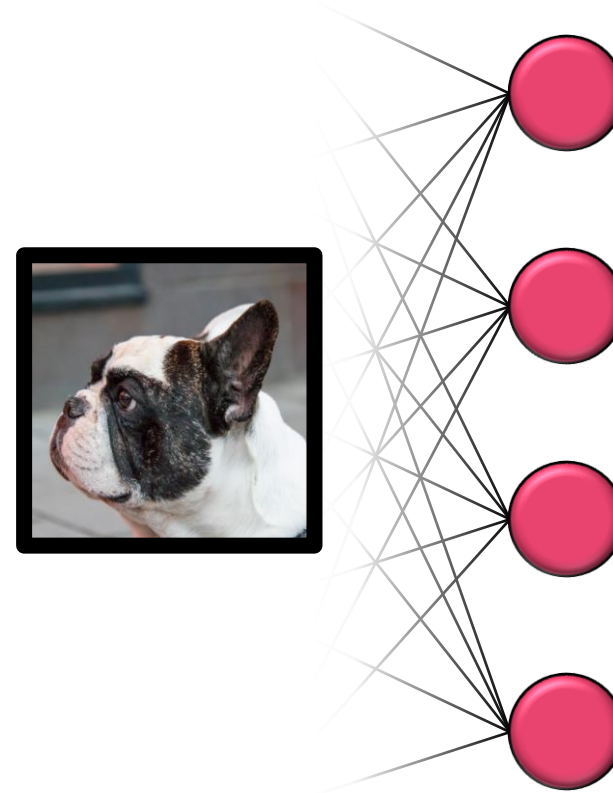
- ❑ Goal
 - **Predict one of two classes.** Class #1 is in general the one to “detect”
 - e.g., “not spam” / “spam”, “positive behavior” / “negative”, “Authentic” / “Fake”...
- ❑ Output value
 - Probability $\in [0; 1]$
 - To determine class 0 or 1, a **threshold** is used (0.5 by default)
- ❑ Output layer
 - **1 neuron**
 - Activation: **Sigmoid**

“Hello world”



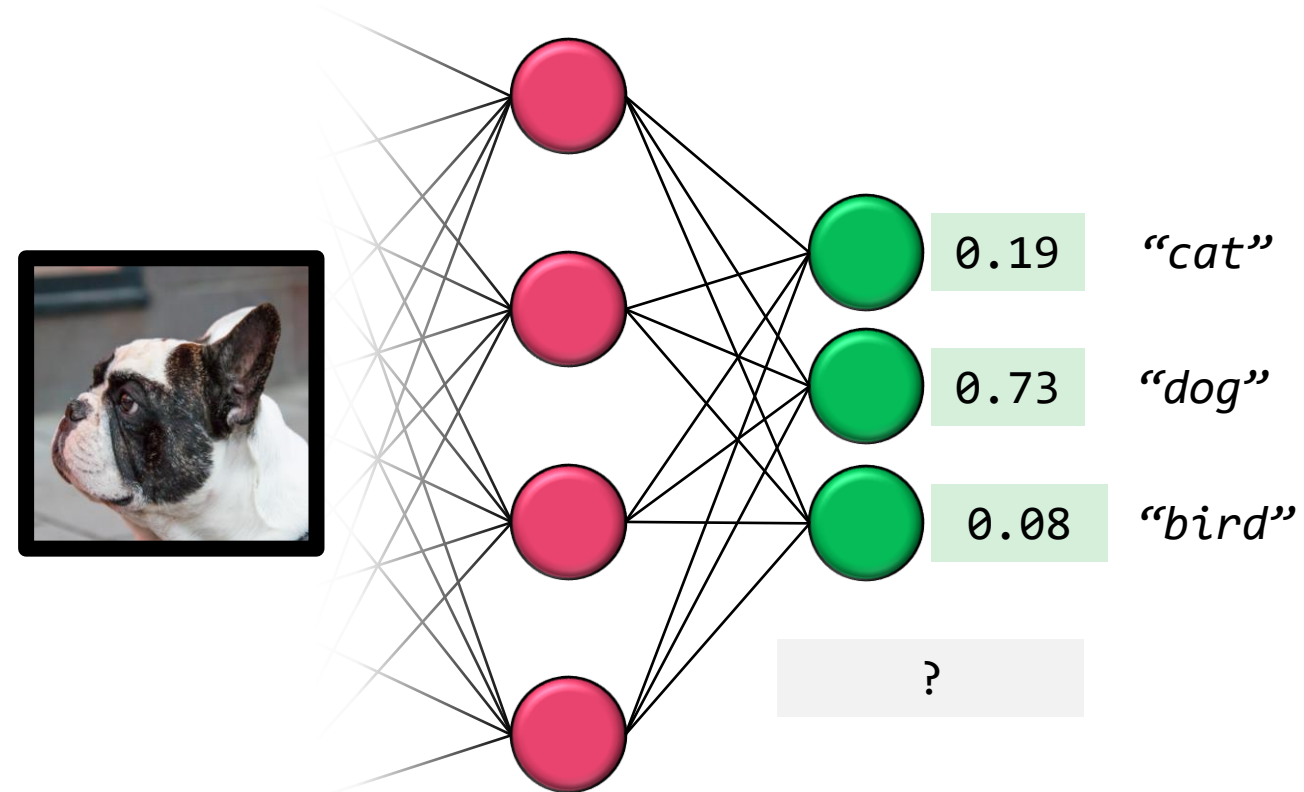
Multiclass Classification

- ❑ Goal
 - **Predict one class among N possible categories**
 - e.g., “cat” / “dog” / “bird”..., a word over a dictionary...
- ❑ Output value
 - ?
- ❑ Output layer
 - ? neurons



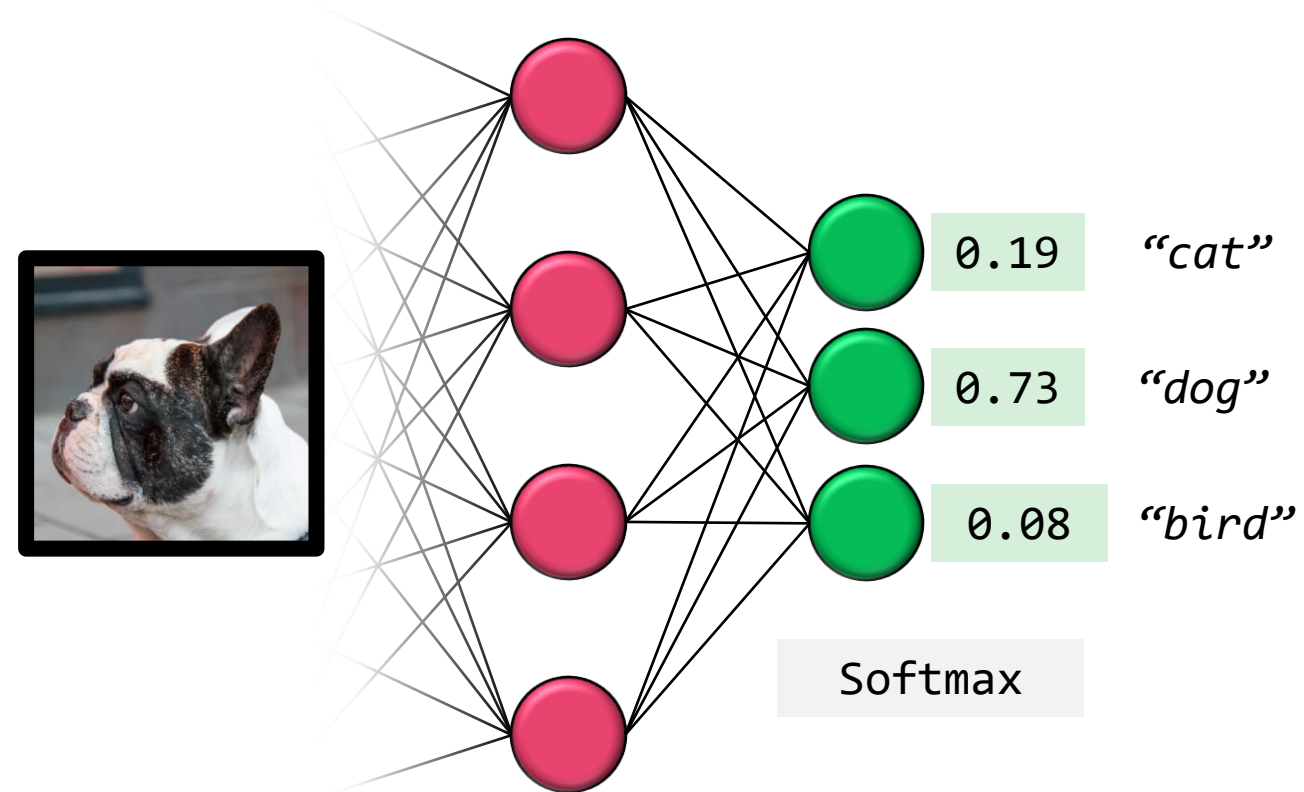
Multiclass Classification

- ❑ Goal
 - **Predict one class among N possible categories**
 - e.g., “cat” / “dog” / “bird”..., a word over a dictionary...
- ❑ Output value
 - Vector of probabilities that sum to 1
- ❑ Output layer
 - **N neurons**
 - Activation: ?



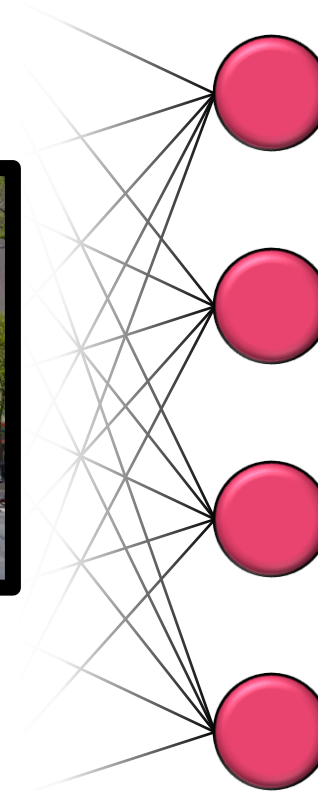
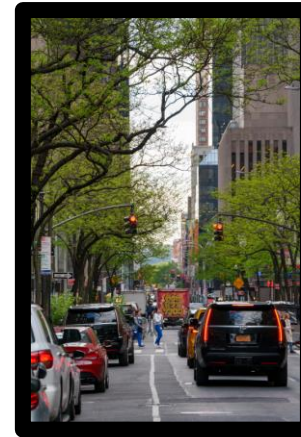
Multiclass Classification

- ❑ Goal
 - **Predict one class among N possible categories**
 - e.g., “cat” / “dog” / “bird”..., a word over a dictionary...
- ❑ Output value
 - Vector of probabilities that sum to 1
- ❑ Output layer
 - **N neurons**
 - Activation: **Softmax**



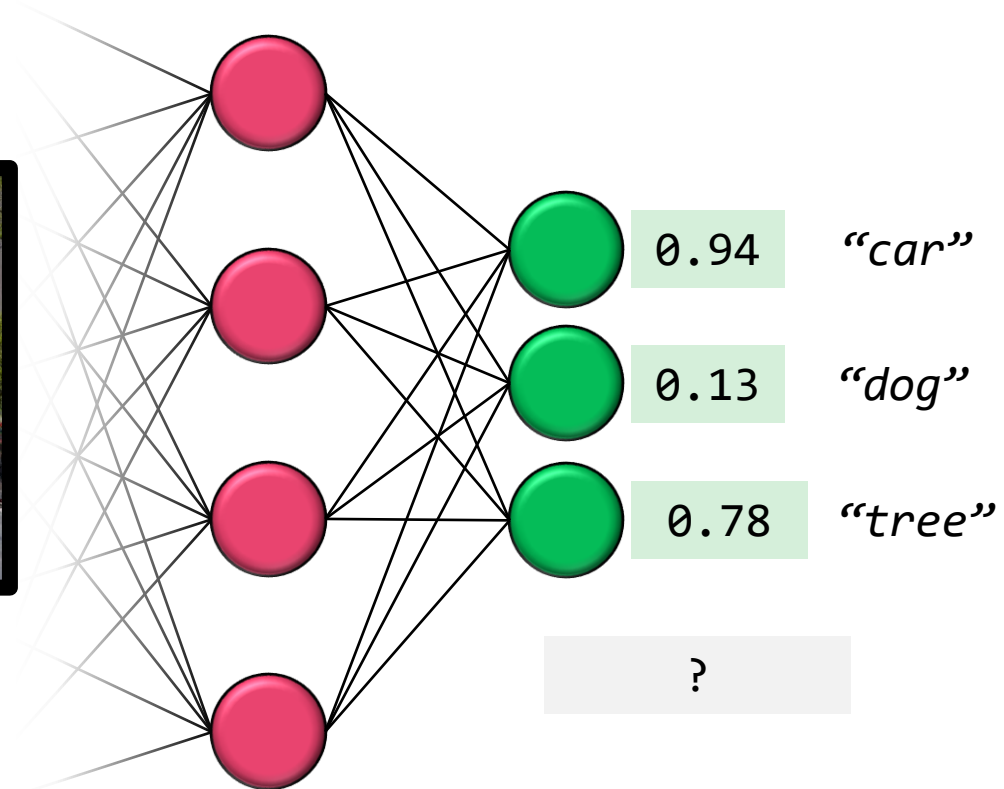
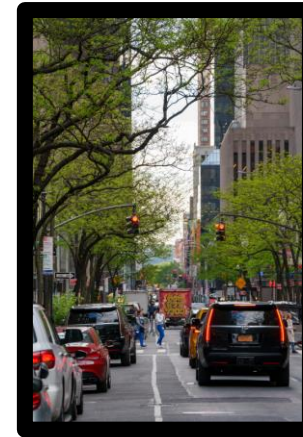
Multilabel Classification

- ❑ Goal
 - **Predict multiple independent labels at once**
 - e.g., [*“car”*, *“human”*, *“traffic light”*] in a photo
- ❑ Output value
 - ?
- ❑ Output layer
 - ? neurons



Multilabel Classification

- ❑ Goal
 - **Predict multiple independent labels at once**
 - e.g., [“car”, “human”, “traffic light”] in a photo
- ❑ Output value
 - Each neuron outputs an **independent probability** between 0 and 1
- ❑ Output layer
 - **N neurons**
 - Activation: ?



Multilabel Classification

- ❑ Goal
 - **Predict multiple independent labels at once**
 - e.g., [“car”, “human”, “traffic light”] in a photo
- ❑ Output value
 - Each neuron outputs an **independent probability** between 0 and 1
- ❑ Output layer
 - **N neurons**
 - Activation: **Sigmoid**

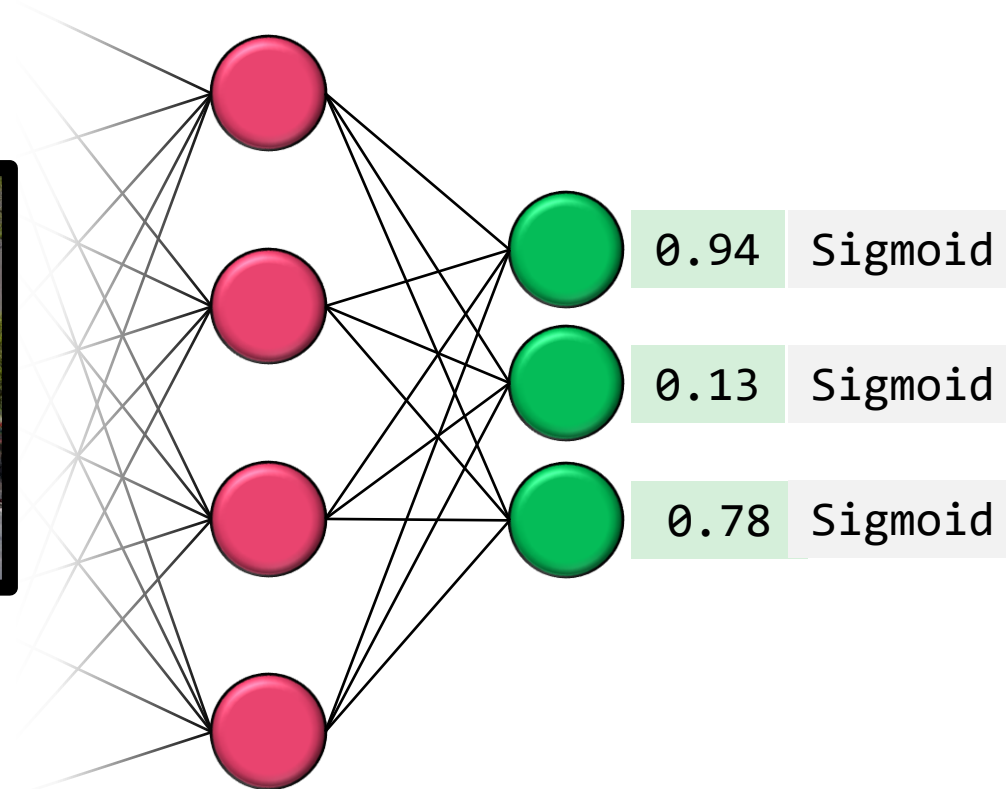
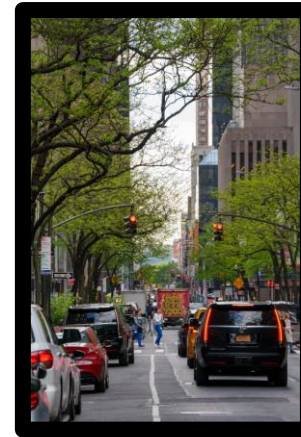


Photo by [Fatih Turan](#) from Pexels